

Prague University of Economics and Business
Faculty of Informatics and Statistics



Loosely Symmetric Neural Network Implementation

BACHELOR THESIS

Study program: Applied Informatics

Field of study: Applied Informatics

Author: Lucie Formánková

Supervisor: doc. Ing. Tomáš Kliegr, Ph.D.

Prague, May 2021

Acknowledgements

I would like to express my gratitude to my thesis supervisor, doc. Ing. Tomáš Kliegr, Ph.D., for his valuable and constructive suggestions.

Finally, I wish to thank my family for their support and encouragement throughout my study.

Abstrakt

Nedostatečná nebo nevyrovnaná data jsou častým problémem např. ve zdravotnictví, kde jsou data omezena kvůli ochraně osobních údajů a soukromí. Takovýto nepoměr má za následek snížení výkonnosti klasifikátorů, protože často dávají přednost klasifikaci majoritní třídy.

Umělá neuronová síť při učení vyvažuje velké množství dobře připravených vyvážených dat. Naproti tomu lidé dokážou získat novou informaci i z mála dat. Tato práce se zabývá implementací kognitivních zkreslení do neuronových sítí, díky které by měl model více napodobovat lidský způsob učení. Model loosely symmetric neural network navržený Taniguchi et al. už byl dříve otestovaný autory v jazyce R a dosáhl nejlepších výsledků z ostatních testovaných. Protože implementace není dostupná, během této práce došlo k jejímu vytvoření v jazyce Python.

Práce obsahuje teoretický úvod do strojového učení, popis algoritmu učení neuronových sítí a loosely symmetric modelu včetně vytvořených variant, a evaluaci. Během evaluace na problematice klasifikace spamu, nová implementace LSNN dokázala držet krok s modelem LSNB a dokonce měla lepší výsledky než model eLSNB, který byl dříve vyhodnocen na stejném datasetu jako nejlepší. Způsob „vylepšení“, ale není v původním popise jednoznačně vysvětlen. Není proto jasné, zda je model správnou implementací původního návrhu modelu LSNN. Tato implementace LSNN modelu dokázala svůj potenciál, proces učení by ale mohl být dále vylepšen doplněním metody předčasného ukončení trénování.

Klíčová slova

umělá neuronová síť, kognitivní zkreslení, nevyvážená data, spam

Abstract

Insufficient or imbalanced data are a common issue, e.g., in healthcare where the data is limited to protect personal data and privacy. This imbalance results in classifiers' poor performance as they usually prioritize the majority class during classification.

The artificial neural network requires a large amount of well-prepared balanced data. Human beings, on the other hand, can extract new information even from a few. This thesis deals with the implementation of cognitive biases into neural networks, which should help the model imitate human learning. The loosely symmetric model designed by Taniguchi et al. had been tested before by authors in the language R and achieved the best results from all tested models. During this thesis, the implementation was created in Python as there is no other available.

The thesis contains a theoretical introduction to machine learning, the fit algorithm of the neural network, the loosely symmetric model description, including created variants, and the evaluation. During the evaluation of the spam classification, the new LSNN implementation managed to keep up with the model LSNB and even achieved better performance than the eLSNB model, which performed best on the same dataset in previous research. However, the method of "enhancement" is not clearly explained in the original description. Therefore, it is unknown whether this model is the correct implementation of the original design of the LSNN model. This implementation of the LSNN model proved its potential. However, the fit process could be further extended with the method of early stopping.

Keywords

artificial neural network, cognitive biases, imbalanced data, spam

Contents

Introduction	13
1 Machine learning	15
1.1 Supervised learning	15
1.2 Unsupervised learning	15
1.3 Reinforcement learning	16
1.4 Spam filtering and machine learning	16
1.5 Preliminaries	16
1.5.1 Python	17
1.5.2 Jupyter Notebook	17
1.5.3 Google Colab	17
1.5.4 NumPy	17
1.5.5 Natural Language Toolkit (NLTK)	18
1.5.6 Scikit-learn	18
1.5.7 Pyplot	18
1.5.8 Pandas	18
2 Neural Networks	19
2.1 Perceptron	19
2.2 Multi-Layer Neural Network	19
2.2.1 Learning process	21
2.2.2 Forward propagation	21
2.2.3 Backpropagation	23
2.2.4 Training	24
2.2.5 Early stopping	24
3 Loosely Symmetric Neural Network	27
3.1 Motivation	27
3.2 Cognitive biases	27
3.3 Loosely Symmetric model	27
3.4 Loosely Symmetric Neural Network implementation	29
3.4.1 LS model definition	30
3.4.2 Complications in implementation	31
4 Text data preprocessing	33
4.1 Noise removal	33
4.2 Lowercasing analyzed text	33
4.3 Tokenization	33
4.4 Normalization	34
4.5 Bag-of-words model	35

4.5.1	CountVectorizer	35
4.6	TF-IDF	37
5	LSNN variants	39
5.1	Description of LSNN variants	39
5.1.1	Variants using the output of the LS model in the change of the weight	39
5.1.2	Variants using the output of the LS model to adjust the value of nodes	39
5.1.3	Variants with a flattened adjustment of the value of nodes	40
5.2	Breast Cancer Wisconsin (Original) Data Set	40
5.3	Variants evaluation	41
5.3.1	Results	41
6	Evaluation	45
6.1	Experimental settings	45
6.1.1	Confusion matrix	46
6.1.2	Accuracy	46
6.1.3	Precision	47
6.1.4	Recall	47
6.1.5	F-measure	47
6.1.6	Imbalanced data set	47
6.1.7	SpamAssassin data set	48
6.1.8	Enron data set	48
6.2	Experiment 1 - Balanced training set	48
6.3	Experiment 2 - Imbalanced training set (real ratio)	49
6.4	Experiment 3 - Imbalanced training set (extreme ratio)	49
6.5	Summary of the evaluation	50
	Conclusions	55
	Bibliography	57
A	Full results of experiment 1 - Balanced training set	63
B	Full results of experiment 2 - Imbalanced training set (real ratio)	65
C	Full results of experiment 3 - Imbalanced training set (extreme ratio)	67

List of Figures

2.1	XOR - truth table and plot	20
2.2	Linear vs. nonlinear problems	20
2.3	Sigmoid function	21
2.4	Neural Network	22
3.1	Relation between $LS(q \setminus p)$ and $LS(p \setminus q)$, and relation between $LS(q \setminus p)$ and $LS(\neg q \setminus \neg p)$	28
5.1	LSNN variants - part 1	42
5.2	LSNN variants - part 2	43
6.1	Experimental settings, source: author	46
6.2	Experiment 1 Metrics - Balanced training set	50
6.3	Experiment 2 Metrics - Imbalanced training set (real ratio)	51
6.4	Experiment 3 Metrics - Imbalanced training set (extreme ratio)	52
6.5	Experiment 1 Results - Balanced training set	52
6.6	Experiment 2 Results - Imbalanced training set (real ratio)	53
6.7	Experiment 3 Results - Imbalanced training set (extreme ratio)	53
A.1	Experiment 1 - Accuracy per training set size	63
A.2	Experiment 1 - F-Measure per training set size	63
A.3	Experiment 1 - Precision per training set size	63
A.4	Experiment 1 - Recall per training set size	63
B.1	Experiment 2 - Accuracy per training set size	65
B.2	Experiment 2 - F-Measure per training set size	65
B.3	Experiment 2 - Precision per training set size	65
B.4	Experiment 2 - Recall per training set size	65
C.1	Experiment 3 - Accuracy per training set size	67
C.2	Experiment 3 - F-Measure per training set size	67
C.3	Experiment 3 - Precision per training set size	67
C.4	Experiment 3 - Recall per training set size	68

List of Tables

3.1	Contingency table, $q \setminus p$	28
5.1	Breast Cancer Wisconsin (Original) Data Set	40
6.1	Confusion matrix	46
6.2	SpamAssassin Public Corpus	48
6.3	Enron dataset	48

List of abbreviations

AI Artificial Intelligence	NLTK Natural Language ToolKit
CP Conditional Probability	REPL Read–Eval–Print Loop
eLSNB enhanced Loosely Symmetric Naive Bayes Classifier	RF Random Forest
FN False Negative	RS Rigidly Symmetric model
FP False Positive	S Symmetric bias
GPU Graphics Processing Unit	SMOTE Synthetic Minority Oversampling Technique
HTML Hypertext Markup Language	SVM Support Vector Machines
LS Loosely Symmetric	TF-IDF Term Frequency–Inverse Document Frequency
LSNB Loosely Symmetric Naive Bayes	TN True Negative
LSNN Loosely Symmetric Neural Network	TP True Positive
MS Middle Symmetry model	VM Virtual Machine
MX Mutually eXclusive bias	XOR eXclusive OR
NLP Natural Language Processing	
NN Neural Network	

Introduction

Artificial neural networks are a robust category of machine learning algorithms that mimics brain neurons to make predictions based on data. Unlike humans, who can learn new things from a small amount of data, neural networks experience overfitting and poor performance when trained on small or imbalanced datasets. Thus, obstacles in areas where it is difficult to obtain enough data led to the creation of more human-like models.

This thesis aims to implement one of these methods - Loosely Symmetric Neural Network. It is a new algorithm based on the utilization of mutually exclusive and symmetric cognitive biases. Although the cognitive biases are opposing standards of logic, they represent the typical human tendency to symmetrize information.

In this thesis, neural networks are first set in the machine learning context. The following parts are focused on understanding the architecture of feedforward neural networks and the implementation of base neural network in the Python language, which is further enhanced with the incorporation of the loosely symmetric model. In the end, the implementation of LSNN is evaluated on the spam classification task, which uses text data. Therefore, the text data preprocessing procedure is also explained. The evaluation results are compared to the performance of neural network without enhancement and the modification of naive Bayes with the LS model.

1. Machine learning

Machine learning is a subset of artificial intelligence (AI). One of the goals of AI is to equip technical systems with the ability to learn based on experience and adapt to changing conditions (1). The ability to learn is considered to be a sign of intelligence (1).

In terms of self-learning algorithms, machine learning can be used to process a massive amount of data, determine hidden connections between features, and make sense of data. Machine learning is also known as the ability to acquire own knowledge by extracting patterns from unprocessed data (2). It solves tasks too difficult to solve by fixed programs created by a human (2). Thanks to machine learning, we can use voice recognition software, powerful search engines, digital assistants, artificial opponents in games, and more.

Machine learning systems can be divided into different types based on whether they are trained with human supervision (supervised, unsupervised, semisupervised, reinforcement learning) (3).

1.1 Supervised learning

During supervised training, models are trained on labeled historical data in order to predict future events. Thus, the system is trained with information about the desired behavior (1).

Data preparation for supervised learning is expensive, but it is valuable during the training as the model's output can be compared to the labeled result.

The typical usage is the classification as spam classification and breast cancer classification, which are further detailed in the practical part of this thesis (see chapters 5 and 6).

Algorithms used for supervised learning include Neural networks, Decision Trees, Random Forests, Linear and Logistic Regression. Neural networks are described in more detail in chapter 2.

1.2 Unsupervised learning

In unsupervised learning, the system does not use labeled or classified data but tries to learn without a "teacher" instead. The system is not able to figure out the correct output but can describe hidden structures from unlabeled data. A typical example is clustering.

1.3 Reinforcement learning

In reinforcement learning, the system (agent) improves its performance based on interactions with the environment and is rewarded with feedback (reinforcement signal) rating its execution (4). This method of bonuses and penalties is used in systems where the goal is to learn some behavior or activity (1).

1.4 Spam filtering and machine learning

In the context of unwanted, obtrusive, unnecessary information which suppresses expected messages, the word *spam* is known since Monty Python's Flying Circus sketch set shown in 1970 (5). The word *spam* is derived from the words "spiced" and "ham" (5).

Nowadays, *Spam* is an anonymous, unsolicited bulk email that has become a significant problem on the internet (5). Not only it takes up everyone's time and space, but it can also be dangerous as users fall victim to fraudulent emails from sources asking for sensitive personal information while pretending to be other companies or institutions. *Ham*, on the other hand, is a non-spam message.

Many big email providers such as Gmail and Outlook have incorporated machine learning techniques into their spam filters. Google uses its machine learning open-source framework TensorFlow. It fights against unwanted messages using rule-based filters and machine learning to find new patterns and flag suspicious messages. In addition, every sender's email address is checked whether it has been blacklisted before. In 2019, Google declared that they block more than 100 million additional spam messages every day, thanks to TensorFlow (6). These messages include image-based messages and messages with hidden embedded content.

Machine learning methods allow spam filters to adapt to new spam attempts. Also, it helps to prevent non-spam messages from being incorrectly flagged as it considers every potential signal.

1.5 Preliminaries

This section is dedicated to the description of technologies used in this thesis to create a machine learning model, its evaluation, and data set preprocessing. This practical usage is further detailed in other chapters.

1.5.1 Python

Python (7) is an object-oriented, interpreted programming language that incorporates classes, modules, and dynamic data types. It supports exceptions and dynamic typing (type can be associated with run-time values). The syntax is clean and easy to read.

Unlike compiled and statically typed languages, Python often has inferior performance in computational tasks. On the other hand, Python is extensible in C and C++. Some of the modules and libraries mentioned below utilize this possibility and achieve better computational performance than the comparable functionality written in Python.

Python is portable - it runs on Linux, macOS, and Windows.

1.5.2 Jupyter Notebook

Jupyter Notebook (8) (also known as IPython Notebook) is an interactive computational environment that supports code execution, text, media, mathematics, plots, and more.

It is shown directly in a web browser as REPL (read-eval-print loop), which means that each input from a user can be immediately evaluated with direct output.

1.5.3 Google Colab

Google Colaboratory (9) is a product from Google Research that is a hosted Jupyter notebook service that provides free access to computing resources, including graphics processing units (GPU). It is applicable to machine learning, data analysis, and education tasks.

Limitations of resources vary over time (maximum VM lifetime, GPU types available, etc.). Colab notebooks are stored in Google Drive (cloud-based storage developed by Google) and can also be loaded from GitHub.

The latest supported Python version in Google Colab is Python 3.7.10.

1.5.4 NumPy

NumPy (10) is a powerful library that benefits from the computational power of language C and the clean and elegant look of Python's syntax.

One of the most important neural network parts is its weights and layers, which are represented as matrices. NumPy offers a fundamental object: N-dimensional array object ndarray, a collection of items indexed using N integers (11). Ndarray has its shape and the kind of item that it is composed of (11). NumPy's N-dimensional array is a homogeneous collection of the same data type; therefore, every item takes up the same size block of memory (11).

NumPy also supports functionality for matrices multiplication, sum, iteration, and other essential operations for neural network implementation.

1.5.5 Natural Language Toolkit (NLTK)

NLTK (12) is a suite of libraries written in Python that offers tools to work with text data like stemming, classification, parsing, and tagging (see Chapter 4). NLTK supports research and teaching in Natural Language Processing (NLP).

1.5.6 Scikit-learn

The Scikit-learn (13) started as a Google Summer of Code project and had its first public release in 2010. Scikit-learn is mainly written in Python and uses NumPy for linear algebra operations. Some core algorithms are written in Cython (compiled language used to create modules designed to perform similar to language C).

This thesis uses Scikit-learn implementation of Neural networks and evaluation methods.

1.5.7 Pyplot

Pyplot is an interface from Matplotlib library (14) and offers a collection of functions used to create and style plots. All plots in this thesis were made using Pyplot.

1.5.8 Pandas

Pandas (15) is a tool for data analysis and manipulation. It offers functionality such as reading and writing data, data alignment and handling missing data, merging and joining datasets. It is flexible and easy to use and optimized for performance because the core is written in Cython and C.

2. Neural Networks

The artificial neural network (NN) is a machine learning method that mimics brain neurons' work to achieve the most accurate predictions. A biological neural circuit is a network of neurons that communicate (pass impulses) through connections called a synapse.

In 1949, Donald Hebb extended neuropsychology knowledge in his book *The organization of behavior* (16). The assumption is that when one neuron fires another neuron repeatedly or persistently, the first neuron's axon enlarges synaptic knobs between them (16). Alternatively, the synaptic knob is reduced when two neurons do not fire for a certain period (16). A synaptic knob is the round ending of the axon and a place where a signal is sent to another neuron, and an axon is a part of a neuron that transfers nerve impulses (17).

2.1 Perceptron

The perceptron is a one neuron model enabling the classification of only linear problems as a binary classifier (18). This model was created by Franck Rosenblatt in 1958. The perceptron was initially designed as a model of the visual system containing three layers. Retina, the first layer, receives information from the environment, which is transformed through receptors and association area into output (1).

The perceptron is incapable of implementing the XOR function (19) and requires linearly separable data (4). Samples in Figure 2.1b represent the nonlinear XOR problem - they can not be separated by a linear function as it is in Figure 2.2.

2.2 Multi-Layer Neural Network

Multi-Layer Neural Network commonly consists of three layers - the input layer, the hidden layer, and the output layer. Each layer has at least one node. A neural network with multiple hidden layers is also referred to as a *deep* artificial neural network (4). The number of nodes in the hidden layer can vary depending on the used activation function and other variables. However, many cases proved that a minimum number of hidden units is required to achieve accurate results (20). Extra hidden units above this number do not dramatically affect accuracy (20). It only takes more time during training.

In a feedforward neural network, connections between the nodes do not form a cycle. Thus, neurons are not connected within the same layer, but each neuron is connected to neurons in the neighbor layers (see Figure 2.4).

This section contains code examples taken from the implementation created during this

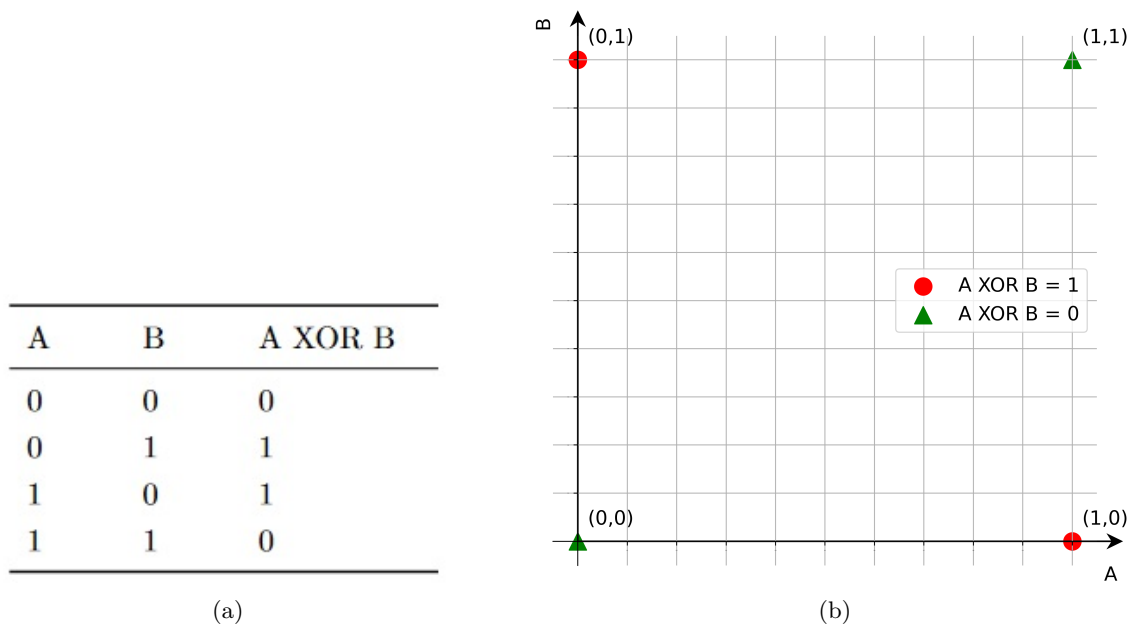


Figure 2.1: XOR: (a) truth table, (b) plot, source: author

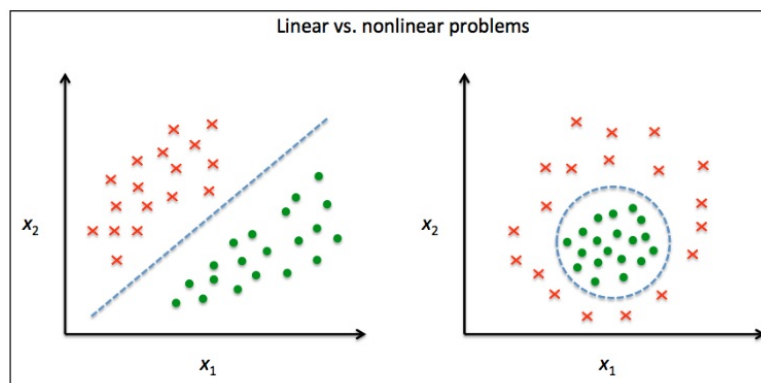


Figure 2.2: Linear vs. nonlinear problems (4)

thesis. All examples were modified to function outside the neural network class. Following implementation does not incorporate bias nodes.

2.2.1 Learning process

The learning process consists of three steps (4). First, the input is forward propagated through each layer to generate output. Then an error is calculated based on the desired result. In the third step, the error is backpropagated in order to update the model and its weights. All three steps are repeated for a number of epochs.

2.2.2 Forward propagation

Forward propagation is used during model training and after fit to predict the class of sample. The neural network's input is transformed through the activation function to achieve non-linearity. The activation function can technically be any function as long as it is differentiable (4). However, the used function should be computationally efficient because it is calculated many times during the fitting process. In order to solve complex problems, a nonlinear function like a sigmoid is needed (4). Sigmoid has a characteristic S-shaped curve (see Figure 2.3) and returns values in the range 0 to 1:

$$\Phi(z) = \frac{1}{1 + e^{-z}} \quad (2.1)$$

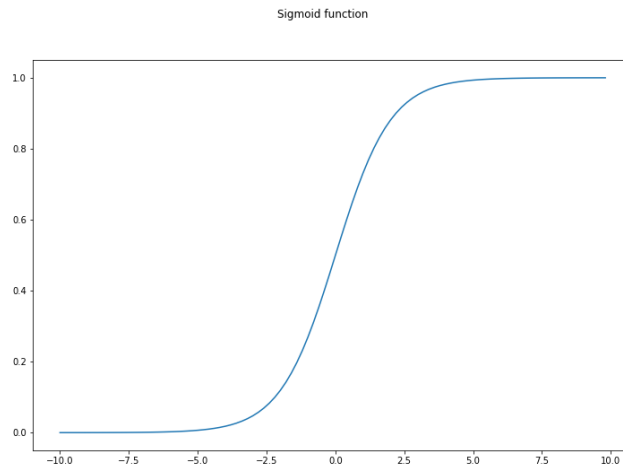


Figure 2.3: Sigmoid function, source: author

This thesis is dealing with a three-layered feedforward neural network. Figure 2.4 depicts a three-layered neural network with three input nodes, four nodes in the hidden layer, and one output as it is a binary classifier.

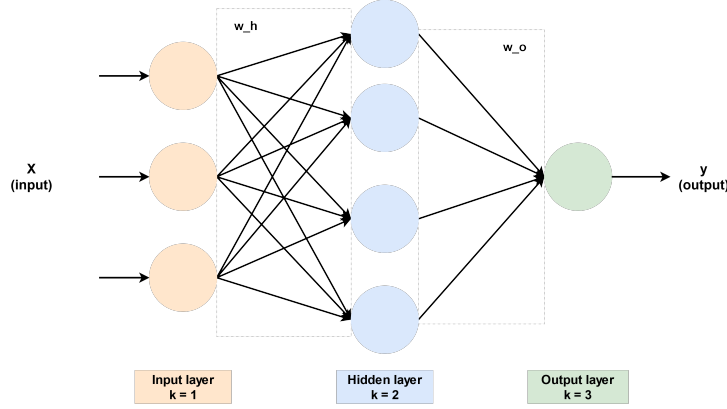


Figure 2.4: Neural Network, source: author

The weights, connections between neurons in neighbor layers, are usually set randomly at the start of training. One of the ways of weights initialization is Xavier initialization, which aims to create weights so that the variance of the activations is the same across every layer. The implementation of Xavier initialization is shown in Listing 2.1.

```

1 import numpy as np
2 n_input = 3 #the number of nodes in the input layer
3 n_hidden = 4 #the number of nodes in the hidden layer
4 n_output = 1 #binary classifier has one output
5
6 def initialize_weights():
7     #weights between input and hidden layer
8     w_h = np.random.randn(n_input, n_hidden)
9     / np.sqrt(n_hidden)
10
11     #weights between hidden and output layer
12     w_o = np.random.randn(n_hidden, n_output)
13     / np.sqrt(n_output)
14
15     return w_h, w_o

```

Listing 2.1: Weight initialization

The input X forms the matrix $A^{(1)}$. The product of $A^{(1)}$ with weights between input and hidden layer $W^{(1)}$ forms weighted input $Z^{(2)}$, further activated into $A^{(2)}$. Next, the activation of the sum of $A^{(2)}$ and $W^{(2)}$ results in the $A^{(3)}$, the output y of the neural network.

$$Z^{(2)} = A^{(1)} \cdot W^{(1)} \quad (2.2)$$

$$A^{(2)} = \Phi(Z^{(2)}) \quad (2.3)$$

The same process is repeated to obtain output values (See Listing 2.2).

```

1 import numpy as np
2 #w_h = weights between input and hidden layer, initialized before
3 #w_o = weights between hidden and output layer, initialized before
4 X = np.array([[1, 2, 4], [3, 4, 8]]) #example
5
6 def sigmoid(z):
7     return 1.0 / (1.0 + np.exp(-z))
8
9 def feedforward(X):
10     # input
11     a1 = X
12
13     # weighted sum - hidden layer
14     z2 = a1.dot(w_h)
15     a2 = sigmoid(z2)
16
17     # weighted sum - output layer
18     z3 = a2.dot(w_o)
19     a3 = sigmoid(z3)
20     return a1, a2, a3, z2, z3

```

Listing 2.2: Feedforward algorithm

Two notations are used in this thesis - the matrix notation used in the implementation ($A^{(2)}$ represents $a2$ in Listing 2.2) and the notation taken from Taniguchi et al. (21), which is later used to describe the backpropagation process and the text regarding the loosely symmetric (LS) model. Thus, applied to equations 2.2, and 2.3, $x^1 = A^{(1)} \cdot W^{(1)}$, $y_1^1 = \frac{1}{1+e^{-x_1^1}}$ and $W^{(1)} = w^{1,2}$. Weights W are in matrix notation indexed in the same way as is activation A and weighted sum Z . $w_{j,i}^{k-1,k}$ is weight from j -th node in $(k-1)$ th layer connected to i -th node in the layer k .

2.2.3 Backpropagation

Backpropagation is the third step in the training process (see section 2.2.1). The process is different for weights between the hidden and output layers and between the input and hidden layers. The aim of the backpropagation in m -layered NN is to update weights and decrease the sum of distances δ_i^m between the network's output y_i^m and the expected true value t_i (see Sum-of-squares error function E in 2.4).

$$E = \frac{1}{2} \sum_i (y_i^m - t_i)^2 = \frac{1}{2} (\delta_i^m)^2 \quad (2.4)$$

$$\Delta w_{j,i}^{k-1,k} = -\alpha \delta_i^k y_i^k (1 - y_i^k) y_j^{k-1} \quad (2.5)$$

First, the error is calculated in the output layer, enabling the error calculation for neurons in the hidden layer. Then the weight $w_{j,i}^{k-1,k}$ of every connection between neurons is adjusted

using the error (see equation 2.5). α is a learning rate, k signifies the order of layer ($k = 1$ is an input layer).

The complete fit function used during training includes iteration over the training set, calculating network output using feed-forward learning, and updating weights through backpropagation, is shown below (see Listing 2.3). In this version, the weights are modified for every training example in each epoch - this is also known as the online learning. An epoch is one cycle of training.

2.2.4 Training

Each dataset and each neural network implementation requires different settings. Training for too long can result in overfitting the training dataset and provide poor performance on test data (3). On the other hand, too short training will also deliver poor performance as the neural network will be under-fitted (3).

Therefore, it is helpful to continuously evaluate the performance during the fitting process and dynamically decide on early stopping when the validation set performance decreases. It is beneficial to evaluate with data that the model has not seen before (4).

2.2.5 Early stopping

The model can be either trained multiple times with different training epochs (which is inefficient and time-consuming) or dynamically controlled for performance drop through generalization error check to decide on an early stop (22). The generalization error is the average error on a validation set (23). When the error on the validation set is higher than it was the last time, the training process should be interrupted, and weights should be assigned to values from the previous run (23).

Early stopping procedures could be repeated multiple times on randomly selected portion of the data set to find the optimal number of epochs and train the model again with all available data (22).

```

1 import numpy as np
2 alpha = 0.5 #learning rate
3 epoch = 100
4
5 def delta_output(output, target):
6     return -(target - output) * sigmoid_derivative(output)
7
8 def delta_hidden(delta_output_times_w_h, output):
9     return sigmoid_derivative(output) * delta_output_times_w_h
10
11 def sigmoid_derivative(x):
12     return x * (1 - x)
13
14 def fit(X_train, y_train):
15     #fit function requires weights w_h and w_o initialized before
16     for i in range(epoch):
17         #for all training instances
18         for j in range(X_train.shape[0]):
19             # target for actual input
20             target = y_train[j]
21
22             # trained input
23             X = X_train[j]
24
25             # activations (a3 is a prediction for sample X)
26             a1, a2, a3, z2, z3 = feedforward(X)
27
28             # backpropagation
29             #adjusting output layer weights
30             adj_o = np.zeros((n_hidden, n_output))
31
32             delta_output_val = delta_output(a3, target)
33             adj_o = - alpha * delta_output_val * a2
34
35             #adjusting hidden layer weights
36             adj_h = np.zeros((n_hidden, n_input))
37
38             #for each hidden node
39             for index in range(n_hidden):
40                 delta_output_times_w_h = w_o[index] * delta_output(
41                     a3, target)
42                 delta_hidden_val = delta_hidden(
43                     delta_output_times_w_h, a2[index])
44                 adj_h[index] = - alpha * delta_hidden_val * X
45
46                 w_o += adj_o.reshape(w_o.shape)
47                 w_h += adj_h.T

```

Listing 2.3: Fit function with backpropagation

3. Loosely Symmetric Neural Network

3.1 Motivation

NN-based models often require a large amount of balanced training data to ensure accuracy, which is not always possible for various reasons. E.g., medical data are protected for privacy issues. The need for classifiers, which can provide accurate results even with small training datasets, led to creating new NN-based models.

3.2 Cognitive biases

Cognitive biases depart from standards of logic. Individuals draw inferences or adopt beliefs without sufficient reason (24). This behavior helps us to understand the design of the mind. Human beings can learn from a small amount of biased data and do not experience overfitting; therefore, more human-like learning models can produce better results.

Two well known cognitive biases were used to develop the loosely symmetric (LS) model - the symmetric bias (S) and the mutually exclusive (MX) bias. When $p \rightarrow q$ is valid, it can be assumed that $q \rightarrow p$ and $\neg p \rightarrow \neg q$ are true as well. An example of this behavior can be found in everyday actions. For example (25), a mother often tells her child *"if you clean up your room, you will be allowed to play."* He also understands the sentence as *"if you don't clean up your room, you will not be allowed to play"* (S), and *"only if you clean up your room, you will be allowed to play"* (MX).

Cognitive symmetries distinguish humans from animals and machines. The human child tends to symmetrize the conditional relations. Human beings have to unconsciously control symmetry intensity because excessive symmetry in thinking is wrong and usually connected to mental disorders and insanity (25).

Implementing these two biases can fasten decision making. LS model has shown a high correlation with human-like inferences (25).

3.3 Loosely Symmetric model

The following table 3.1 displays the co-occurrence between two events where p is a cause, and q is an effect resulting from the presence of cause. The four cells in the table are the frequencies of these co-occurrences, which can be used to derive joint probabilities ($P(p,q)$, $P(p,\neg q)$, $P(\neg p,q)$, $P(\neg p,\neg q)$) (25). $q \setminus p$ means that q occurs when p has happened. Applied to neural networks, p signifies that "neuron x sent a strong signal," and q represents that

"neuron y was activated" (21).

Table 3.1: Contingency table, $q \setminus p$

	q	¬q
p	a	b
¬p	c	d

The conditional probability (CP) model and rigidly symmetric (RS) model had only a weak correlation with data sets from human experiments and are not appropriate to interpret human cognition (25). Middle symmetry (MS) model with parameters α and β controls symmetries - $MS_{\alpha,\beta}$. α and β can take values from interval $[0, 1]$.

$MS_{0,0}$ is identical to CP, and $MS_{1,1}$ is the same as RS:

$$MS_{\alpha,\beta}(q \setminus p) = \frac{a + \beta d}{a + \beta d + b + \alpha c}, \quad (3.1)$$

$$RS(q \setminus p) = MS_{1,1}(q \setminus p) = \frac{a + 1d}{a + 1d + b + 1c} = \frac{a + d}{a + d + b + c}, \quad (3.2)$$

$$CP(q \setminus p) = MS_{0,0}(q \setminus p) = \frac{a}{a + b}. \quad (3.3)$$

Shinohara et al. discovered that setting parameters α and β as $\alpha = CP(p \setminus q) = \frac{a}{a+c}$; $\beta = CP(p \setminus \neg q) = \frac{b}{b+d}$ works well to adjust symmetries like S and MX biases. This model is called the loose symmetry (LS) model:

$$LS(q \setminus p) = \frac{a + \frac{b}{b+d}d}{a + \frac{b}{b+d}d + b + \frac{a}{a+c}c} = \frac{a + \frac{bd}{b+d}}{a + \frac{bd}{b+d} + b + \frac{ac}{a+c}}. \quad (3.4)$$

LS model achieves independent and flexible adjustment of S and MX biases (25). When both α and β equals zero, LS model becomes equivalent to the CP model.

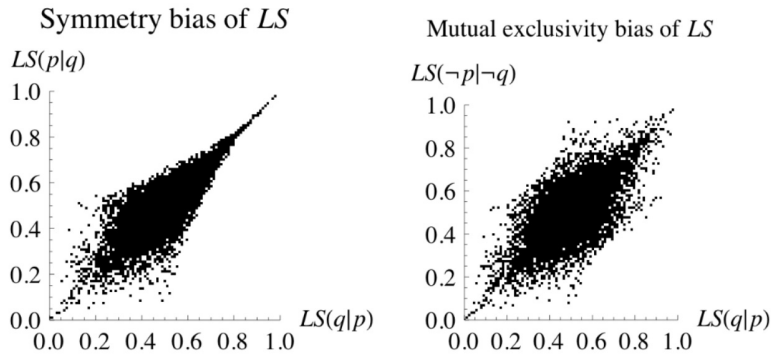


Figure 3.1: Relation between $LS(q \setminus p)$ and $LS(p \setminus q)$, and relation between $LS(q \setminus p)$ and $LS(\neg q \setminus \neg p)$ (25)

Figure 3.1 shows the visualization of the biases. Each graph contains ten thousand points randomly generated by uniformly taking parameters a , b , c , and d from $[0, 1]$ (25). If $b = c$, the LS model satisfies symmetric bias completely (21). In addition, if also $a = c$, the LS model satisfies both symmetric and mutually exclusive biases (21). LS model does not always wholly represents these biases but instead displays the intermediate states of S and MX . Otherwise, the model would be strongly illogical and would not show similarity to a human deduction (21). Graphs in Figure 3.1 shows shapes on the border of proportional and random distribution (21).

LS model was successfully implemented as an extension of Naive Bayes classifier, creating loosely symmetric naive Bayes (LSNB) (26, 27). Later on, it was utilized in neural networks to adjust value of nodes and bring higher accuracy. This model is called loosely symmetric neural network (LSNN) (21).

The loosely symmetric model enhances the neural network's performance by adding a process to layer inputs during training (21). LS model attempts to realize more flexible Hebbian learning (16, 21). LSNN can omit and revive nodes, which is a more adaptable approach. LSNN showed the best overall performance when evaluated on the breast cancer classification task (21). Other models used during experimentation were NN, SVM, RF, NN with dropout, and NN with batch normalization.

3.4 Loosely Symmetric Neural Network implementation

LSNN is based on a three-layered (input layer, one hidden layer, and output layer) feedforward neural network with a sigmoid function commonly used for binary classification.

The procedure of LSNN consists of three steps (21):

1. Compute value of nodes through a feed-forward algorithm.
2. Adjust the value of nodes in the $(k - 1)$ th layer.
3. Update weights using adjusted value of nodes (see equation 3.5).

$$\Delta w_{j,i}^{k-1,k} = -\alpha \delta_i^k y_i^k (1 - y_i^k) LS(y_j^{k-1}) \quad (3.5)$$

In order to apply LS model to the neural network, parameters a , b , c , and d , which reflect the degree to which the nodes were or were not activated, have to be set (21):

$$a = y_i^{k-1}, \quad (3.6)$$

$$b = 1 - y_i^{k-1}, \quad (3.7)$$

$$c = 1 - x_j^k, \quad (3.8)$$

$$d = x_j^k. \quad (3.9)$$

Although the LS model needs both parameters a and d to be assigned, and $LS(y_j^{k-1}, x_j^k)$ seems more appropriate than $LS(y_j^{k-1})$, the equation had been left in the original form defined by Taniguchi et al (21).

3.4.1 LS model definition

LS model should be able to predict whether the node assigned to a contributed to the activation of the node assigned to d . (21). Listing 3.1 shows the modified LS model implementation used in the LSNN implementation, where $a = y^{k-1}$ and $d = x_j^k$. Therefore, parameter b is not a scalar number but rather a result in the deduction of the vector of ones and vector a . This modification resulted in fewer iteration steps, which affects the speed of the training process.

```

1 import numpy as np
2
3 def loosely_symmetric(a, d):
4     b = np.ones((a.shape)) - a
5     c = 1 - d
6
7     bd = (b*d) / (b+d)
8     ac = (a*c) / (a+c)
9
10    # 1 = a + b
11    return (a + bd) / (1 + ac + bd)

```

Listing 3.1: LS model function

Listing 3.2. shows the usage of the LS model during the backpropagation process. Here, the parameter d is the classifier's output $z3$ before activation, and a is matrix $a2$ which is a result of the activated product of weights and input values. The usage of the variable ls would precede the weights adjustment calculation on line 20 in Listing 2.3.

Another difference between equation 3.4 and Listing 3.1 is that the sum of parameters a and b is replaced with constant 1 because $b = 1 - a$ (see parameters assignments in 3.4).

```

1 ls = loosely_symmetric(a2, z3)

```

Listing 3.2: Output layer calculation

As the implementation of the LS model works with matrix y^{k-1} , the following chapters use notation $LS(y^{k-1})$ instead of $LS(y_j^{k-1})$:

$$\Delta w_{*,i}^{k-1,k} = -\alpha \delta_i^k y_i^k (1 - y_i^k) LS(y^{k-1}) \quad (3.10)$$

3.4.2 Complications in implementation

LS model was described in detail in previous researches (21, 25, 26). However, its incorporation into a neural network is not described sufficiently in Taniguchi et al. (21) in order to guarantee accurate implementation.

Equation 3.5 replaces y_j^{k-1} with $LS(y_j^{k-1})$. The procedure of the LSNN contains steps adjusting the value of nodes and refers to this exact equation. However, $LS(y_j^{k-1})$, as in 3.4, returns a value that estimates whether y_j^{k-1} contributed to the activation of x_j^k and that y_j^{k-1} should be enhanced if it did. How the enhancement work is not further detailed.

Settings of experiments with original implementation by Taniguchi et al. (21) does not contain all parameter setting (e.g., learning rate) which would help during comparison with another implementation. The architecture of their base neural network is also not detailed enough.

4. Text data preprocessing

A large amount of unstructured or semi-structured data is generated every day (e.g., emails, reviews, social media posts), but unlike structured datasets with categories and values, it is not easy to conclude from text sources without further processing.

Preprocessing transforms raw data into a form that is usable and analyzable for a specific task. The form has to be digestible for machine learning algorithms.

It is divided into four stages: data cleaning, integration, reduction, and transformation. Text preprocessing includes tokenization, lower casing, stop words removal, stemming, and lemmatization.

4.1 Noise removal

Raw text data can often contain formatting marks such as HTML tags. Noise removal is about removing special characters, symbols, or words that can misrepresent the meaning or not benefit the analysis.

4.2 Lowercasing analyzed text

Words at the start of a sentence usually start with an uppercase letter but begin with a lowercase letter when they are in the middle of the sentence. When analyzing the occurrence of each word, words with different casing are considered to be different. Thus, it is convenient to lowercase all terms in the analyzed text when it is not desired to differ between words with different casing.

4.3 Tokenization

Every text consists of words that carry the information. To learn the meaning of the text, we read groups of words that together tell a story. During preprocessing for the natural language processing task, we have to extract each word from the text. This process is called tokenizing, and each word is a token.

Python already has a suitable library called **Natural Language Toolkit** (see Section 1.5.5), which gives several tokenizing options. Examples below shows results of tokenization on the sentence “*These aren’t George’s ponies.*”:

```
1 ['These', 'aren't', 'George's', 'ponies.']
```

Listing 4.1: WhitespaceTokenizer example

```
1 ['These', 'aren', "'", 't', 'George', "'", 's', 'ponies', '.']
```

Listing 4.2: WordPunctTokenizer example

```
1 ['These', 'are', "n't", 'George', "'s", 'ponies', '.']
```

Listing 4.3: TreebankWordTokenizer example

According to run results is clear that `WhitespaceTokenizer` (See Listing 4.1) completely ignores the meaning of special characters, divides tokens by white spaces, and keeps symbols like dots at the end of the word. `WordPunctTokenizer` (See Listing 4.2), on the other hand, breaks sentences into tokens divided by both whitespaces and punctuation.

The third way to tokenize text is to use `TreebankWordTokenizer` (See Listing 4.3). This function uses regular expressions to tokenize text, so for example, *aren't* is divided into *are* and *n't*.

4.4 Normalization

Suffixes make two terms distinguished (not equal) even if they have the same base word. However, two standard options to transform these words to be considered the same can be used.

The first option to normalize these words is the `PorterStemmer` function that trims suffixes, but as seen below, it does not always return the correct form:

```
1 import nltk
2
3 text = "These aren't George's ponies"
4 tokenizer = nltk.tokenize.TreebankWordTokenizer()
5 stemmer = nltk.stem.PorterStemmer()
6 tokens = tokenizer.tokenize(text)
7
8 for token in tokens:
9     print(stemmer.stem(token))
10
11
12 these
13 are
14 n't
15 georg
16 's
17 poni
```

Listing 4.4: PorterStemmer demonstration

Stemming produces roots/bases of words, also known as stems. There are two major errors in stemming, overstemming and understemming (28). Overstemming occurs when the stemmer

produces the same stem for two unrelated words - these words are treated the same in further processing. Understemming, on the other hand, is the opposite issue. Several words that come originally from the same root are treated as different due to different stemmer's output.

The other option is to find a dictionary form known as a lemma. This is a function of `WordNetLemmatizer` from the Natural Language Toolkit library. `WordNetLemmatizer` can return confusing results when it is not given the category of the word as a part-of-speech (pos) tag. As manual tagging of each word with the correct category is inefficient and lengthy, there is an alternative in the form of a built-in function `pos_tag`, that can assign part of speech tags automatically.

4.5 Bag-of-words model

Bag-of-words represent text as a feature vector and it is created in two simple steps (4):

1. Create vocabulary of unique tokens from the entire dataset. When n is defined, the vocabulary considers top max features and is ordered by term frequency - only n most common tokens are selected.
2. Create a feature vector from each document that contains the number of how many times each word occurred in it.

This thesis uses implementation from scikit-learn library (see Section 1.5.6) - `sklearn.feature_extraction.text.CountVectorizer`.

4.5.1 CountVectorizer

Listing 4.6 shows the specific settings of `CountVectorizer` class for the spam classification task. The following description describes its parameters and their options.

```
1 import nltk
2
3 def tokenizeData(text):
4     tokenizer = nltk.tokenize.WordPunctTokenizer()
5     stemmer = nltk.stem.PorterStemmer()
6
7     text = text.lower()
8     tokens = tokenizer.tokenize(text)
9     #root words
10    stems = [stemmer.stem(token) for token in tokens]
11    return stems
```

Listing 4.5: Tokenizer

```

1 from sklearn.feature_extraction.text import CountVectorizer
2
3 vectorizer = CountVectorizer(
4     decode_error='replace',
5     lowercase=False,
6     tokenizer=tokenizeData,
7     stop_words=None,
8     max_df=0.99,
9     min_df=2,
10    max_features=100,
11    binary=False)

```

Listing 4.6: CountVectorizer initialization

`lowercase` is `True` by default, but our custom tokenizer already handles this functionality. Section 4.2 describes lowercasing in more detail.

The parameter `tokenizer` should be set when we want to override the string tokenization step with different approach but still want to preserve Vectorizer's n-gram creation functionality. In this case, function `__tokenizeData` (see Listing 4.5) was used.

`stop_words` are not selected in this case. During text processing, stop words are usually removed to clear text from overused words like the verb "be." In spam classification, this step might do more harm than good. For example, over-usage of words can also indicate unwanted messages.

The parameter `ngram_range` was left with the default value and therefore left out of initialization. N-gram is a contiguous sequence of `n` words and is useful when analyzing text by phrases. The default value is `(1, 1)`. This tuple represents the lower and upper boundary of the range of `n`-values for different word n-grams. For example, `ngram_range=(1, 1)` is a vocabulary of single words called **unigrams**, `(2, 2)` are **bigrams**, and `(1, 2)` are both unigrams and bigrams.

`max_df` and `min_df` represent maximal or minimal document frequency threshold. Words that are outside of this interval are excluded from the built vocabulary. Therefore, `max_df=1.0` means that terms that appear in more than 100% of the documents in the set will be ignored. Thus, this default setting will not ignore any terms. On the other hand, `min_df=2` indicates that terms that appear in less than two documents will also be ignored. Thresholds can be set as a percentage (float) or an exact count (integer).

We can either set a maximum number of vocabulary features according to the term frequency or work with all words. With another parameter - `vocabulary` - it is possible to use different vocabulary instead of creating a new one from given text. But if the `vocabulary` parameter is not set to `None` (default value), the range given in `max_features` is ignored.

`binary=True` determines that all non-zero occurrences are set to 1. This confirms that the word was used in the text but omits the information of how many times. `binary` was set to

False in this case.

4.6 TF-IDF

More frequent words in the resulting vector will have a higher weight than the less occurring. Thus, more relevant terms crucial for classification will have lower importance as a result of the representation according to the number of occurrences. With the technique term frequency-inverse document frequency (TF-IDF), the weight of frequent words can be down weighted.

Scikit-learn (see Section 1.5.6) contains transformer `TfidfTransformer`, which takes a matrix of term/token counts as input and returns normalized tf-idf representation.

TF-IDF is in Scikit-learn defined as:

$$tf-idf(t,d) = tf(t,d) \times idf(t), \quad (4.1)$$

Term frequency, $tf(t,d)$, is the frequency of term t . Document frequency, $df(t)$, is the number of documents d that contain the term t . Term inverse document frequency calculation differs based on the value of `smooth_idf` parameter. By default, `smooth_idf=True`, and the constant 1 is added to the numerator and denominator of the idf (see equation 4.2). This constant represents as if an additional document containing all words exactly once was seen, which prevents divisions by zero.

In the opposite case, the constant 1 is added only to the whole equation (see equation 4.3). This adjustment prevents terms occurring in all documents from excluding as their idf value would be equal to zero.

Scikit-learn documentation states that both versions vary from the standard textbook equation that defines the inverse document frequency as $idf(t) = \log \frac{n}{df(t)+1}$ (13).

$$idf(t) = \log \frac{1+n}{1+df(t)} + 1 \quad (4.2)$$

$$idf(t) = \log \frac{n}{df(t)} + 1 \quad (4.3)$$

The constant n is the total number of documents. The final step is L2-normalization of $tf-idf(t,d)$.

5. LSNN variants

The description of the LSNN classifier (21) is short and full of subjective terms and homonyms with many potential interpretations. It is mainly focused on describing the feed forward algorithm rather than how the enhancement works. Unfortunately, the attempt to get clarification from the authors was not successful.

Therefore, several types of enhancements with the LS model were made and evaluated on the Breast Cancer Wisconsin data set, the same dataset as the original implementation was (21). Using the same dataset was a reasonable way to examine the classifier's behavior and determine whether the result resembled the LSNN implementation of Taniguchi et al. (21). In addition, processing natural language and predicting illness based on a few features are very different tasks. Thus, the classifier might perform well on one task but wrong on another.

Evaluation, in this case, is made to help determine whether the variant (enhancement type) produces usable results. However, it can not be said whether the approach is the same as is the original one (21).

5.1 Description of LSNN variants

All variants of LS model usage in the NN model are compared to the version without enhancement which is marked as `enhancement_type="none"`.

5.1.1 Variants using the output of the LS model in the change of the weight

The first tested variants, `enhancement_type="ls_value"` and `enhancement_type="ls_value_save"`, are based on the definition of weight change using the LS model, as can be seen in Equation 3.10, which means that value $LS(y^{(k-1)})$ is used instead of $y^{(k-1)}$. If the name of the variant contains the word "save," it means that the original $y^{(k-1)}$ value is replaced with the value used in the weight change equation and is used further in backpropagation.

5.1.2 Variants using the output of the LS model to adjust the value of nodes

During the first attempts to create the LSNN model, the direct usage of LS model output in the weight change equation resulted in inferior performance. Therefore, new variants were created.

Taniguchi et al. described that the LS model is able to predict whether the node contributed to the activation of another (21). When the LS model outputs a higher value than the actual

node's value, it probably contributed to the activation and should be enhanced (21). How the enhancement is implemented is not clear from original description.

Therefore, another assumption was that actual nodes value should be increased or decreased by a constant value, percentage, or dynamically according to a weight of a contribution to the activation.

The weight of a contribution comes from the assumption that a higher difference between $y^{(k-1)}$ and $LS(y^{(k-1)})$ means that the node is too undervalued or overvalued. Variants `enhancement_type="save_node_difference"` and `enhancement_type="value_node_difference"` uses this assumption and dynamically increases or decreases $y^{(k-1)}$ by a percentage (based on a training parameter `enhancement`) of the difference of the $y^{(k-1)}$ and $LS(y^{(k-1)})$ model output. Increasing (or decreasing) $y^{(k-1)}$ by the percentage of its value are variants `enhancement_type="save_node_unified"` and `enhancement_type="value_node_unified"`.

5.1.3 Variants with a flattened adjustment of the value of nodes

The last two variants are `enhancement_type="save_node_unified_flattened"` and `enhancement_type="value_node_unified_flattened"`. At some point, the enhancement (or reduction) can be higher than the difference between $y^{(k-1)}$ and $LS(y^{(k-1)})$ - especially when the change is significant. These two variants contain additional checks that replace the $y^{(k-1)}$ with $LS(y^{(k-1)})$ when the increased $y^{(k-1)}$ would exceed $LS(y^{(k-1)})$ (or the decreased $y^{(k-1)}$ would fall below $LS(y^{(k-1)})$).

5.2 Breast Cancer Wisconsin (Original) Data Set

The data set contains nine features describing medical facts that are later used to determine whether the conclusion is benign or malignant (29). Benignity is a harmless condition, malignancy is the opposite.

The target was changed to binary values as it will be further used to evaluate a binary classifier. There are around 700 instances, but 17 were removed as they contained missing values. Benign instances are more represented.

Table 5.1: Breast Cancer Wisconsin (Original) Data Set

Examples	Benign	Malignant
683	444	239
100%	65%	35%

5.3 Variants evaluation

For each version, a k-fold cross validation was performed on the Breast cancer data set. More precisely, each type of classifier was trained ten times for 100 epochs using a different part of the whole dataset, while ten percent were kept for evaluation. Therefore, it was achievable to compare the accuracy between the classification on the known data and the new data. The learning rate was set to 0.5. Taniguchi et al. mentioned the usage of the learning rate parameter (21), but the actual value is unknown. Hidden layer had 30 nodes. After all classifiers were trained for 100 epochs, results from each run were summed and divided by a number of runs to obtain average training accuracy per epoch and average evaluation accuracy per epoch. The accuracy might not be the best metric to evaluate imbalanced datasets because achieving high accuracy is possible even when the model prefers the majority class. However, the accuracy was chosen as a suitable metric for observing model behavior during training as it was unknown whether models will even be able to produce some results.

Each figure in this section shows a graphical representation of the average accuracy's development through the whole training process. Accuracy was measured at the beginning of each epoch on the evaluation set, which contains data unused during the fit process, and the training set, which is a set used for training.

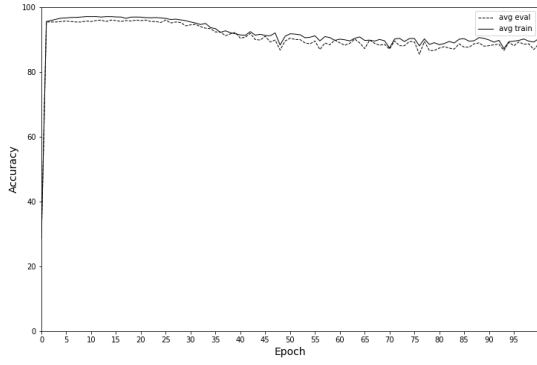
5.3.1 Results

The results (see Figures 5.1c and 5.1d) showed that the LS model output probably is not intended part of the weight adjustment but more likely serves as an indicator designating that the node was significant for the prediction. Both *ls_value* and *ls_value_save* were unable to generalize and produced poor results.

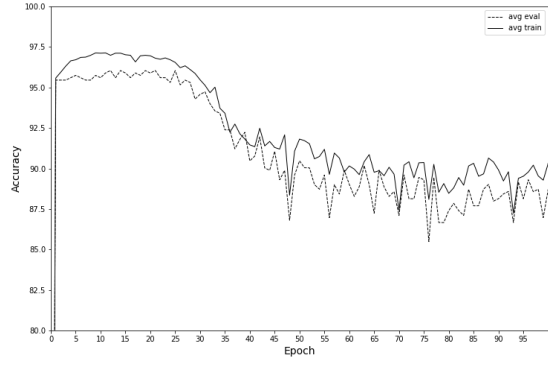
Unified change to $y^{(k-1)}$ value had proven to be the best option of all tested variants (see Figure 5.2). The *value_node_unified* model (see Figure 5.2a) resembled results from training the model without enhancement. The *save_node_unified* model (see Figure 5.2c) had more interesting results than the *value_node_unified* as the accuracy was overall stable and the highest. However, it had some difficulties around the 65th epoch, which the *save_node_unified_flattened* model resolved (see Figure 5.2e).

Models with unified change had the best outcome, but their implementation was the most computationally intensive as the difference between $LS(y^{(k-1)})$ and $y^{(k-1)}$ had to be checked for every value. The breast cancer data set has nine features, but the spam classification data can have thousands of features. Therefore, the implementation was further developed and the speed was optimized. The resulting algorithm was used later in chapter Evaluation.

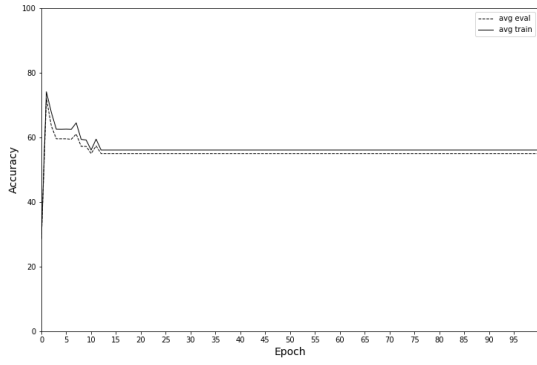
The implementation of LSNN class with mentioned variants without speed optimization can be found on GitHub (<https://github.com/formankova/LSNN>)



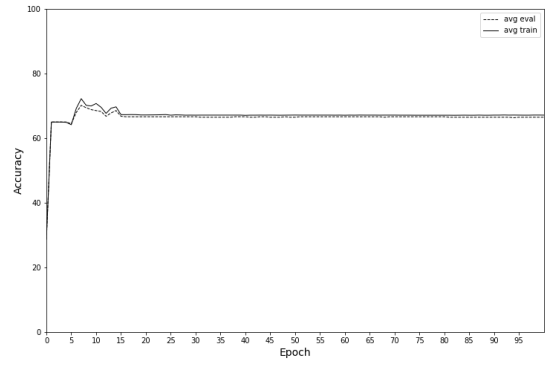
(a)



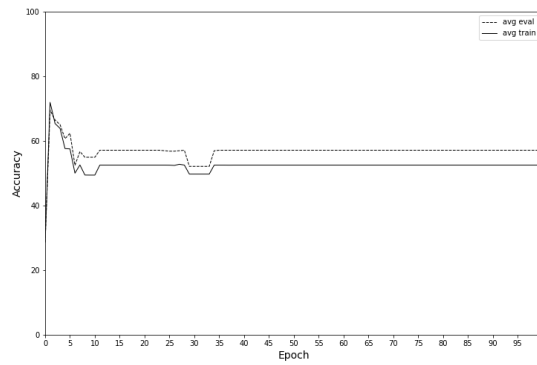
(b)



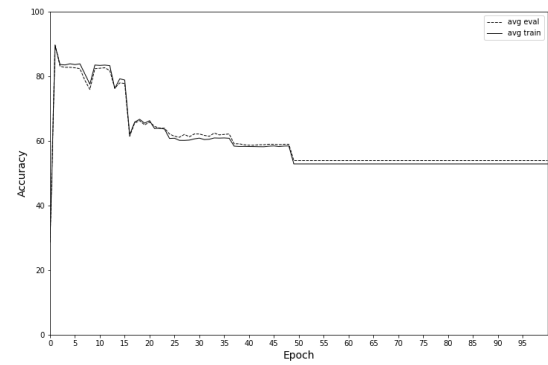
(c)



(d)

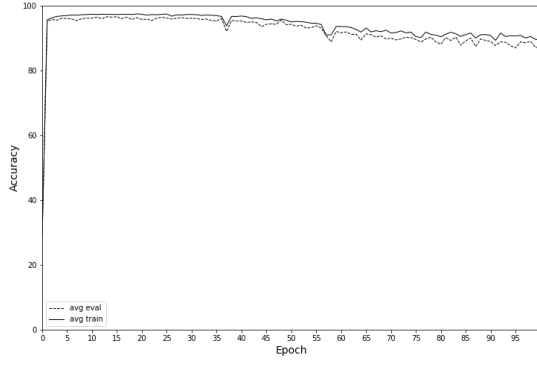


(e)

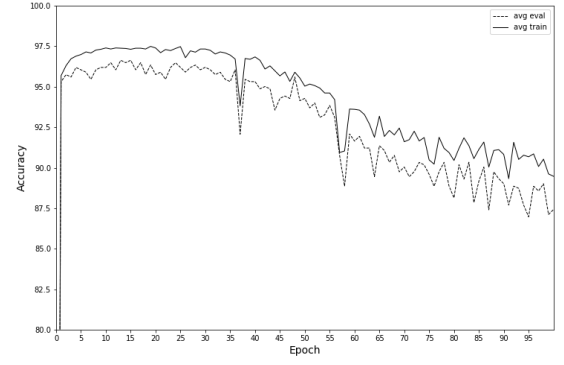


(f)

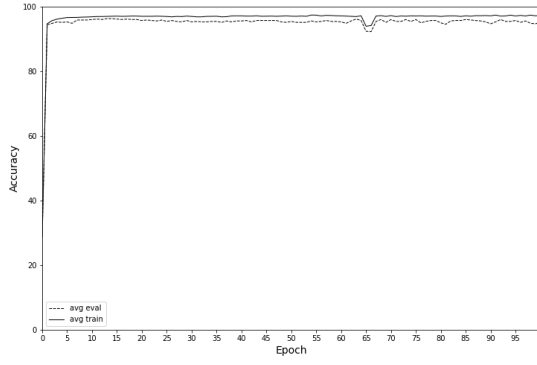
Figure 5.1: LSNN variants: (a) and (b) none, (c) ls_value, (d) ls_value_save, (e) save_node_difference, (f) value_node_difference, source: author



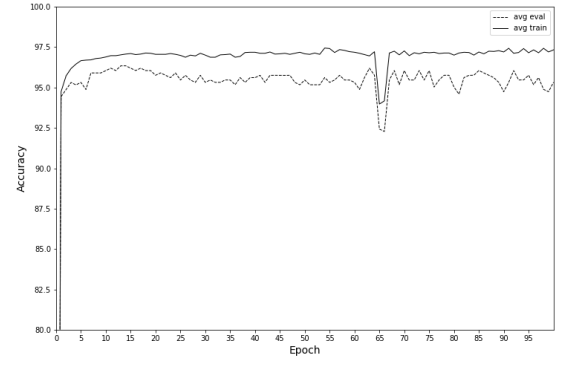
(a)



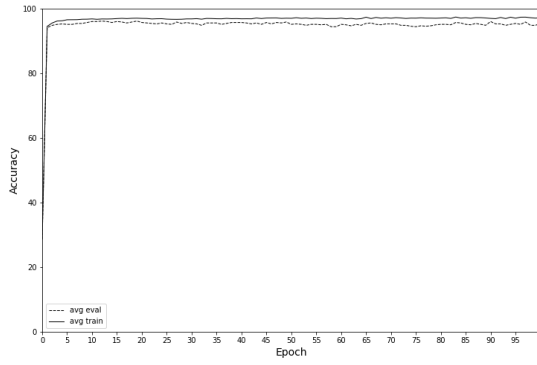
(b)



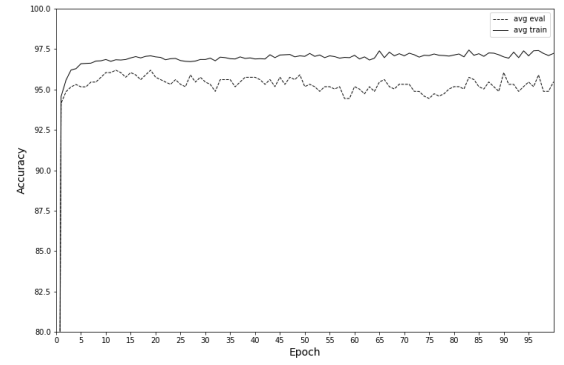
(c)



(d)



(e)



(f)

Figure 5.2: LSNN variants: (a) and (b) value_node_unified, (c) and (d) save_node_unified, (e) and (f) save_node_unified_flattened, source: author

6. Evaluation

In this chapter, the LSNN classifier is evaluated on spam data set and compared with LSBN and NN from the scikit-learn library.

6.1 Experimental settings

For the evaluation, the SpamAssassin public corpus was used. The same data set was used in experiments with the original implementation of the LSBN and eLSNB models (26). The implementation of LSBN and eLSNB by Mareš was evaluated on the Enron data set in his thesis (27). Therefore, both data sets are described, but evaluation in this thesis was performed only on the SpamAssassin data set.

All possible adjustments and early stopping were not used in "basic" NN as LSNN implementation does not use them. The used activation function was sigmoid, and the learning method was online. All neural networks were three-layered nets with ten hidden nodes trained for 100 epochs.

Three experiments were conducted. In the first experiment, the training data set contained a balanced ratio of spam and ham examples. The second experiment kept the original ratio of the SpamAssassin public corpus - 31% spam, 69% ham. Furthermore, the last experiment introduced an even more unbalanced data set - 90% ham, 10% spam. Examples marked as "hard_spam" were removed from the set.

Each experiment was conducted with several sizes of training data sets (see Figure 6.1). For each run, accuracy, precision, recall and F-measure values were recorded using `sklearn.metrics` library. Each experiment had its own testing set of 600 samples. Thus, both model trained on 50 samples and the model trained on 600 samples were evaluated with the same data set that contains only samples, which models did not see before.

LSNN model was tested with settings of learning rate α 0.5 and 0.6. Tested enhancement types were *save_node_unified*, *value_node_unified*, and *save_node_unified_flattened*. In most cases, the enhancement value was 0.05, but the third experiment with the smallest ratio of spam samples introduced an additional LSNN variant, *save_node_unified_flattened* LSNN model with 0.2 enhancement.

Results of each experiment are shown in the sections below in the form of graphical representation. Also, a comparison by average metric is shown. Precise results can be found in Appendices A, B, and C.

		Training set size							
	Class	Ratio	50	100	200	300	400	500	600
Experiment 1	Spam	0.5	25	50	100	150	200	250	300
Balanced training set	Ham	0.5	25	50	100	150	200	250	300
Experiment 2	Spam	0.31	16	31	62	93	124	155	186
Imbalanced training set (real ratio)	Ham	0.69	35	69	138	207	276	345	414
Experiment 3	Spam	0.1	5	10	20	30	40	50	60
Imbalanced training set (extreme ratio)	Ham	0.9	45	90	180	270	360	450	540

Figure 6.1: Experimental settings, source: author

6.1.1 Confusion matrix

A confusion matrix (see Table 6.1) can be used to describe the performance of a classification model. True positive (TP) is a number of correctly classified positive samples. False positive (FP) are all corretly classified negative samples. False negative (FN) is all positive samples classified as negative samples. False positive (FP) is all negative samples classified as positive samples.

Table 6.1: Confusion matrix

		Predicted	
		P	N
Actual	P	TP	FN
	N	FP	TN

Applied to the spam classification task, the positive class is spam, and the negative class is ham. FP is ham samples labeled as spam. FN is spam samples labeled as ham.

6.1.2 Accuracy

Accuracy measures the share that has been classified correctly. Accuracy is not a suitable metric to compare models when the training data set is imbalanced as the high accuracy is achievable by a model that predicts only the majority class.

$$accuracy_score = \frac{tp + tn}{tp + fp + tn + fn} \quad (6.1)$$

6.1.3 Precision

Precision measures the ability of the classifier to not label negative samples as positive. The best value is 1, and the worst is 0.

$$precision_score = \frac{tp}{tp + fp} \quad (6.2)$$

6.1.4 Recall

Recall measures the ability to find all the positive samples. The best value is 1, and the worst is 0.

$$recall_score = \frac{tp}{tp + fn} \quad (6.3)$$

6.1.5 F-measure

F-measure is also known as the F1 score. It is a weighted average of precision and recall. The best value is again 1, and the worst is 0. F-measure is a commonly used metric for imbalanced classification.

$$f1_score = 2 * \frac{precision_score * recall_score}{precision_score + recall_score} \quad (6.4)$$

6.1.6 Imbalanced data set

In practice, imbalanced datasets are commonly modified to create a more balanced set: methods to achieve this include undersampling, oversampling, and synthetic sampling (30).

Undersampling includes the randomized removal of examples from the majority class. The disadvantage is that deleted samples may contain important information which will be lost.

Oversampling increases the number of training examples by duplicating minority class examples. The oversampling may result in overfitting.

Synthetic sampling (SMOTE) is a technique that is used to create similar examples of minority classes synthetically.

6.1.7 SpamAssassin data set

SpamAssassin is an open-source anti-spam platform that also offers a public corpus with 1897 spam messages and 4150 ham messages (31). Two hundred fifty ham messages are marked as "hard_ham" as they use unusual HTML markup, colored text, and phrases often seen in spam messages. This dataset is an excellent example of an unbalanced sample with not many examples. Only 31% is labeled as spam, and the rest is ham.

Table 6.2: SpamAssassin Public Corpus

Examples	Ham	Spam
6047	4150	1897
100%	69%	31%

6.1.8 Enron data set

Enron dataset is more robust than SpamAssassin. Enron Corporation was an American company that went bankrupt in 2002 due to its systematic accounting fraud. During its investigation, personal files from approximately 150 Enron employees were made publicly available and were later used to create e-mail classification benchmarks. Table 6.3 shows the distribution in the Enron dataset, which Mareš used to evaluate his implementation of LSNB and eLSNB (27).

Table 6.3: Enron dataset

Examples	Ham	Spam
33716	16545	17171
100%	49%	51%

6.2 Experiment 1 - Balanced training set

The best overall performance had the neural network without the enhancement with loosely symmetric model. The worst performing model was eLSNB, which is the enhanced loosely symmetric naive bayes classifier. LSNB model had a similar average performance to the LSNN model with tested parameters. The best settings of tested variants of LSNN were learning rate 0.5, enhancement 0.05, and enhancement type *value_node_unified* - it had the second-highest average value of F-measure and accuracy.

The enhanced LSNB model (eLSNB) had the highest precision score, which is considered an essential metric for spam classifiers. On the other hand, it should not be high at the expense

of a recall score, which implies how well the model could classify all positive examples. All other models had a significantly higher recall score.

When we look at complete results per each run (see Appendix A), the LSNB achieved the highest accuracy at the smallest training data set. However, the classification dropped at a training size of 300 samples, and LSNB could not overcome LSNN since. Due to poor performance at the smallest training size, LSNN and LSNB had similar average accuracy. F-measure followed a similar trend - LSNB had an excellent start and started performing worse at the training size of 300 samples. As for the recall metric, LSNB and eLSNB had the worst result at almost every size of the training set.

6.3 Experiment 2 - Imbalanced training set (real ratio)

During Experiment 2 with the imbalanced dataset, neural network without LS model also achieved the best average F-measure. As mentioned in section 6.1.2, the accuracy score should not be the key metric in imbalanced datasets.

Results are relatively similar to Experiment 1. The eLSNB model again achieved a high precision score but the lowest F-measure and recall score. LSNB and LSNN had similar results. Although the LSNN setting with 0.5 alpha and 0.1 enhancement had the lowest average precision score, the average recall score was second highest.

When we look at complete results per each run (see Appendix B), the LSNN model with enhancement type *save_node_unified_flattened* achieved the best recall rate at the smallest data set training size but also had the worst precision score. The LSNB model had the second-best F-measure score until the training size of 500, where the LSNN model replaced his position. The eLSNB model performed the worst at every training size apart from the size of 200 samples. However, its F-measure value at 200 training samples was almost identical to the LSNN model with enhancement type *value_node_unified*.

6.4 Experiment 3 - Imbalanced training set (extreme ratio)

Experiment 3 was the most interesting as the ham samples represented 90% of the whole training set. Figure 6.7 shows that all models (except eLSNB and NN) struggled with F-measure at the smallest training set size. The best performing model was, in this case, the LSNB model. The second-best performing model was the LSNN model, which was in this experiment set with 0.2 (20%) enhancement and the enhancement type was *save_node_unified_flattened*. Higher enhancement had a positive influence on the training in a highly imbalanced data set. NN had an even higher F-measure than LSNN, thanks to the high recall score.

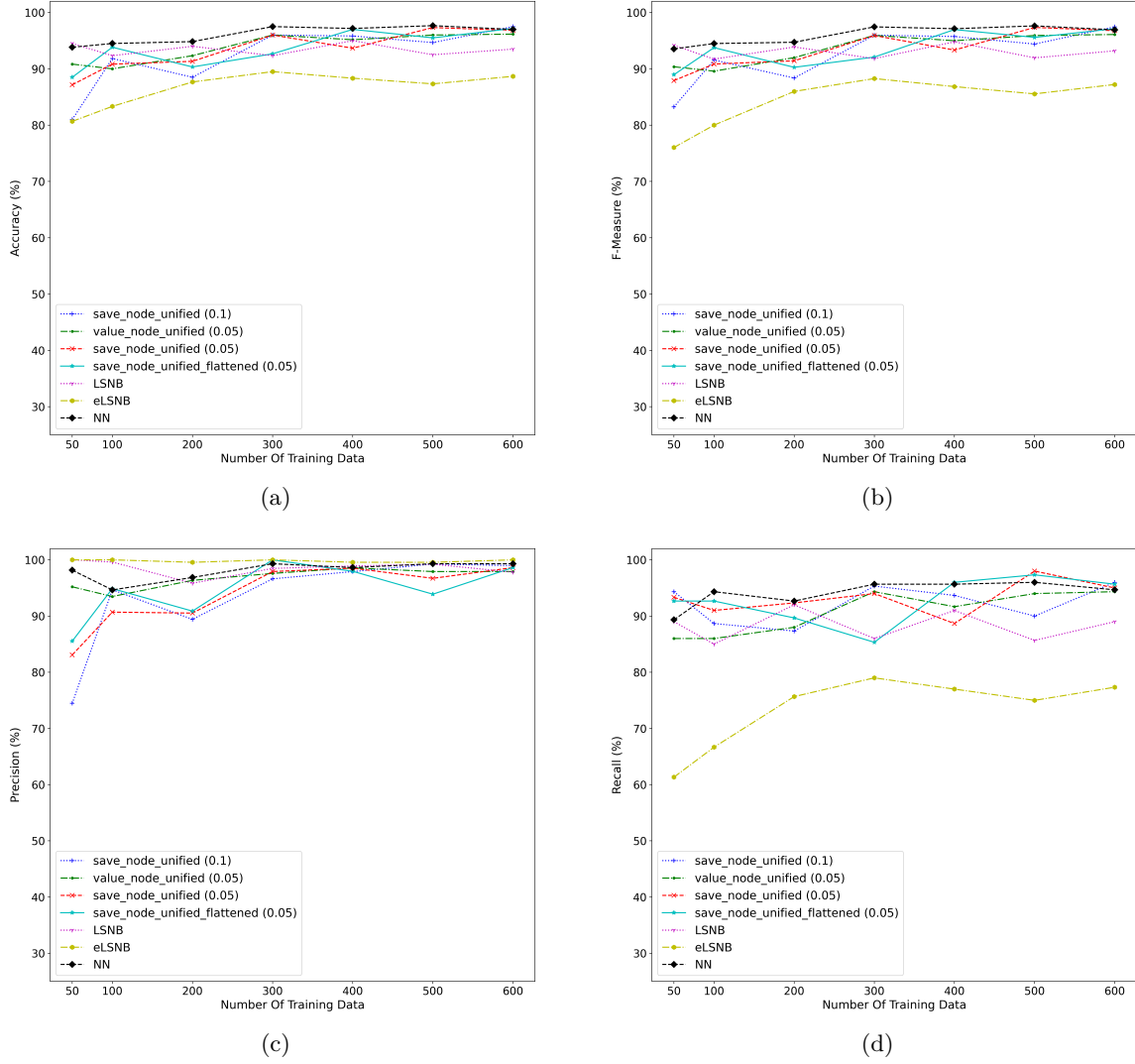
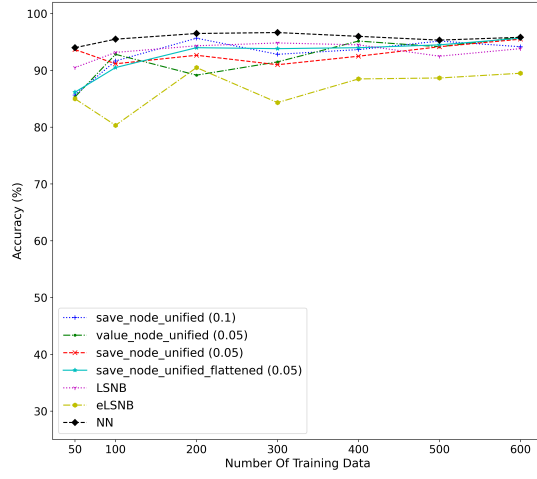


Figure 6.2: Experiment 1 Metrics: (a) classification accuracy, (b) F-measure, (c) precision, (d) recall, source: author

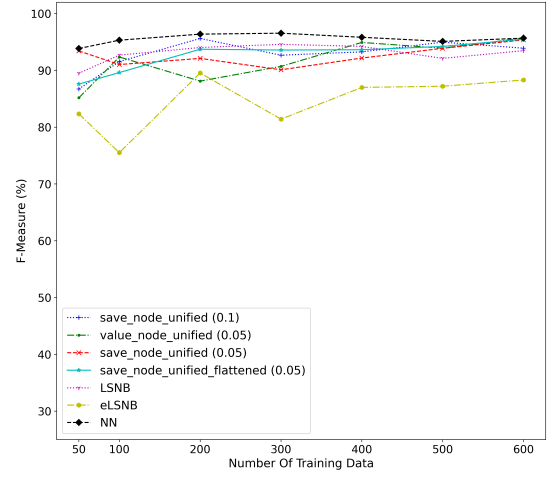
6.5 Summary of the evaluation

The LSNN model achieved similar performance to the LSNB model. However, a neural network set with maximum limitations to resemble the settings of the LSNN without the LS model incorporation had the best results in the first two experiments.

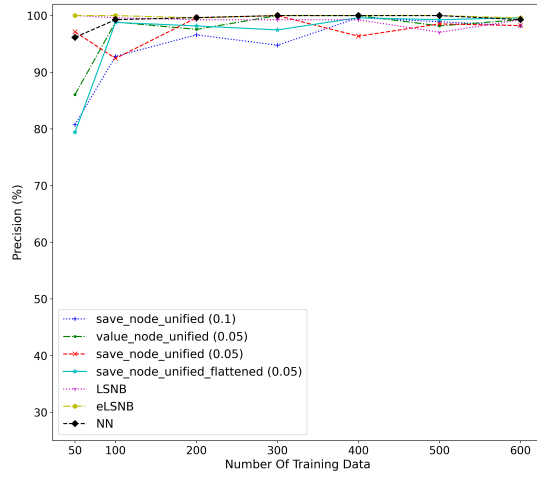
Although the eLSNB model performed best in previous researches, it was the worst performing model in these three experiments. LSNB and eLSNB were initialized with default parameters that their implementation defines. According to examples of usage found in the GitHub repository, these parameter settings were used to create a spam classifier (32). Preprocessing differs from the approach used by Mareš (27) - stop words were not removed. However, LSNB and eLSNB were trained with the same data, and the LSNB performed well. Therefore, the difference between individual experiments might not be in data preprocessing or classifier initialization.



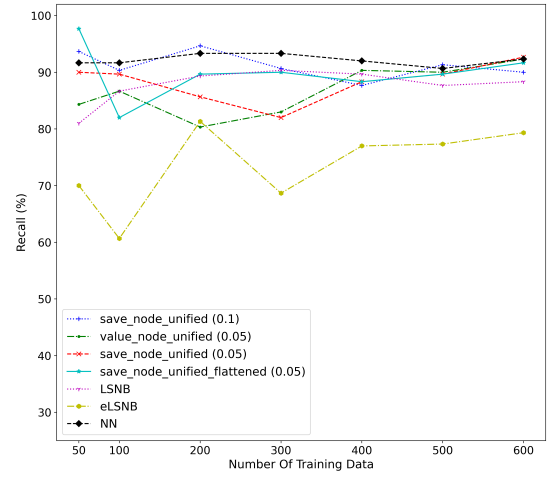
(a)



(b)



(c)



(d)

Figure 6.3: Experiment 2 Metrics: (a) classification accuracy, (b) F-measure, (c) precision, (d) recall, source: author

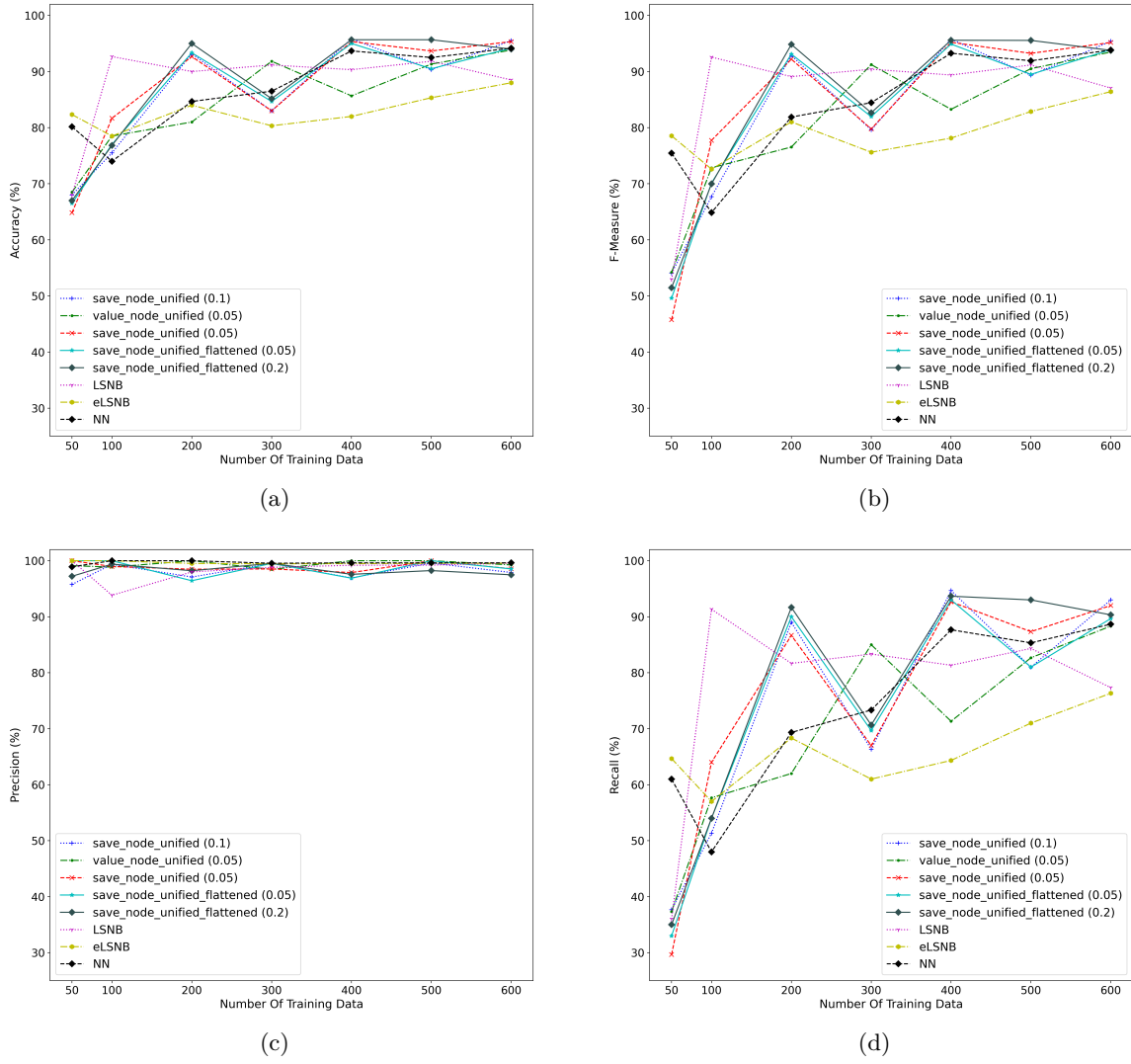


Figure 6.4: Experiment 3 Metrics: (a) classification accuracy, (b) F-measure, (c) precision, (d) recall, source: author

Alpha		Enhancement	Model	Accuracy	F-Measure	Precision	Recall
LSNB	0.5	0.1	save_node_unified	92.19%	92.39%	93.05%	92.19%
	0.5	0.05	value_node_unified	93.79%	93.55%	96.72%	90.62%
	0.6	0.05	save_node_unified	93.31%	93.37%	93.72%	93.19%
	0.6	0.05	save_node_unified_flattened	93.57%	93.54%	94.54%	92.76%
LSNB				93.45%	93.08%	98.55%	88.24%
eLSNB				86.50%	84.27%	99.81%	73.14%
NN				96.07%	95.98%	98.03%	94.05%

Min Max

Figure 6.5: Experiment 1 Results - Balanced training set, source: author

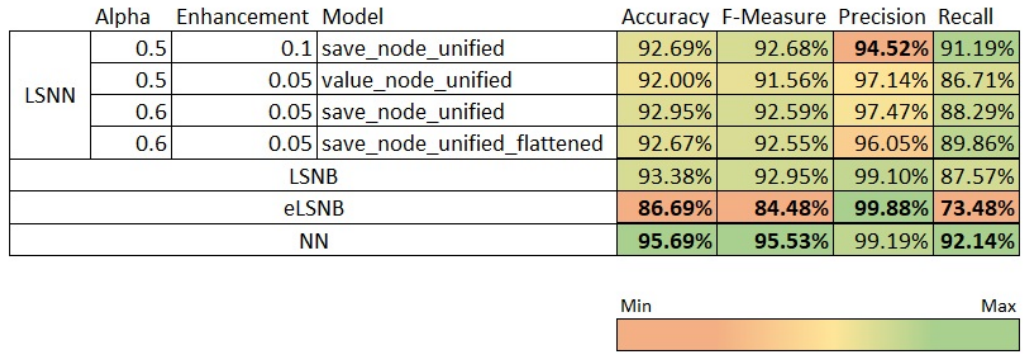


Figure 6.6: Experiment 2 Results - Imbalanced training set (real ratio), source: author

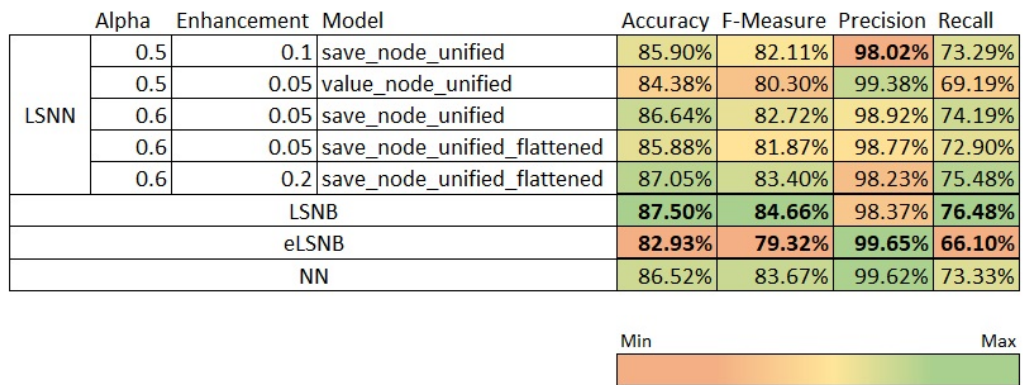


Figure 6.7: Experiment 3 Results - Imbalanced training set (extreme ratio), source: author

Conclusions

The implementation of the LSNN model was difficult due to unclear descriptions of key parts of the algorithm from the authors. Unfortunately, an attempt to get clarification was not successful. Therefore, several possible variants of LSNN were created and evaluated.

Evaluated variants compared to the neural network without enhancement and with the LSNB model produced good results similar to the performance of the LSNB model. In particular, the experiment with highly imbalanced data showed the positive influence of the LS model as the LSNN model achieved a higher average recall rate and, in most cases, a higher F-measure rate than the neural network without enhancement.

It can not be said whether the implementation is correct. However, it proved its potential as it performed similarly to the LSNB model. The aim of this thesis was fulfilled as much as it was possible with limited information. During experiments with possible variants, neural networks with the LS model overcame the variant without enhancement, and the benefit of the LS model was recorded.

The resulting implementation with additional examples was saved to the GitHub repository (<https://github.com/formankova/LSNN>). It is the only public implementation - no other implementation has been found during the search. The algorithm could be further improved by adding methods that directly influence the training process. For example, an early stopping method that helps to prevent overfitting.

Bibliography

1. BERKA, Petr. *Dobývání znalostí z databází*. Academia, 2003. ISBN 978-80-200-1062-9.
2. GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
3. GÉRON, A. *Hands-on machine learning with Scikit-Learn and TensorFlow : concepts, tools, and techniques to build intelligent systems*. Beijing: O'Reilly, 2017. ISBN 978-1-4919-6229-9.
4. RASCHKA, Sebastian. *Python Machine Learning*. Birmingham: Packt Publishing Ltd., 2015. ISBN 978-1-78355-513-0.
5. *What is spam?* [Online] [visited on 2021-05-04]. Available from: <https://encyclopedia.kaspersky.com/knowledge/what-is-spam/>.
6. *Spam does not bring us joy—ridding Gmail of 100 million more spam messages with TensorFlow* [online]. 2019 [visited on 2021-04-16]. Available from: <https://cloud.google.com/blog/products/g-suite/ridding-gmail-of-100-million-more-spam-messages-with-tensorflow>.
7. FOUNDATION, Python Software. *What is Python?* [Online] [visited on 2021-03-27]. Available from: <https://docs.python.org/3/faq/general.html#what-is-python>.
8. PÉREZ, Fernando; GRANGER, Brian E. IPython: a System for Interactive Scientific Computing. *Computing in Science and Engineering*. 2007-05, vol. 9, no. 3, pp. 21–29. ISSN 1521-9615. Available from DOI: 10.1109/MCSE.2007.53.
9. *Colaboratory* [online] [visited on 2021-03-27]. Available from: <https://research.google.com/colaboratory/faq.html>.
10. HARRIS, C.R.; MILLMAN, K.J.; WALT, S.J. van der, et al. Array programming with NumPy. *Nature*. 2020-09, vol. 585, no. 7825, pp. 357–362. Available from DOI: 10.1038/s41586-020-2649-2.
11. OLIPHANT, Travis E. *Guide to NumPy*. Birmingham, 2006.
12. BIRD, Steven; KLEIN, Ewan; LOPER, Edward. *Natural language processing with Python: analyzing text with the natural language toolkit*. O'Reilly Media, Inc., 2009.
13. PEDREGOSA, F.; VAROQUAUX, G.; GRAMFORT, A.; MICHEL, V.; THIRION, B., et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*. 2011, vol. 12, pp. 2825–2830.
14. HUNTER, J. D. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*. 2007, vol. 9, no. 3, pp. 90–95. Available from DOI: 10.1109/MCSE.2007.55.
15. MCKINNEY, Wes. Data Structures for Statistical Computing in Python. In: WALT, Stéfán van der; MILLMAN, Jarrod (eds.). *Proceedings of the 9th Python in Science Conference*. 2010, pp. 56–61. Available from DOI: 10.25080/Majora-92bf1922-00a.

16. HEBB, Donald O. *The Organization of Behaviour: A neuropsychological theory*. New York: John Wiley and Sons, 1949. ISBN 0471367273.
 17. KIMBALL, John W. *Synapses* [online]. 2020 [visited on 2021-05-01]. Available from: <https://bio.libretexts.org/@go/page/5647>.
 18. ROSENBLATT, Frank. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*. 1958, vol. 65, no. 6. Available also from: <http://www.mdpi.com/2227-9709/5/1/2/pdf>.
 19. MINSKY, Marvin L.; PAPERT, Seymour A. *Perceptrons: an introduction to computational geometry*. Cambridge: The M.I.T. Press, 1949. ISBN 0 262 13043 2.
 20. MITCHELL, Tom M. *Machine Learning*. Cambridge: McGraw-Hill, 1997. ISBN 0070428077.
 21. TANIGUCHI, Hidetaka; SATO, Hiroshi; SHIRAKAWA, Tomohiro. Implementation of Human Cognitive Bias on Neural Network and Its Application to Breast Cancer Diagnosis. *SICE Journal of Control, Measurement, and System Integratio*. 2019, vol. 12, no. 2. Available from DOI: doi.org/10.9746/jcmsi.12.56.
 22. BROWNEE, Jason. *A Gentle Introduction to Early Stopping to Avoid Overtraining Neural Networks* [online]. 2018 [visited on 2021-03-27]. Available from: <https://machinelearningmastery.com/early-stopping-to-avoid-overtraining-neural-network-models/>.
 23. ORR, Genevieve B.; MÜLLER, Klaus-Robert. *Neural Networks: Tricks of the Trade*. Heidelberg: Springer-Verlag Berlin Heidelberg, 1998. ISBN 978-3-540-49430-0.
 24. HASELTON, Martie G.; NETTLE, Daniel; MURRAY, Damian R. The Evolution of Cognitive Bias. In: Packt Publishing Ltd., 2015. Available from DOI: [10.1002/9781119125563.evpsych241](https://doi.org/10.1002/9781119125563.evpsych241).
 25. TAKAHASHI, Tatsuji; NAKANO, Masahiro; SHINOHARA, Shuji. Cognitive symmetry: illogical but rational biases. *Symmetry: Culture and Science*. 2010, vol. 21. Available also from: https://www.researchgate.net/publication/285850238_Cognitive_Symmetry_Illogical_but_Rational_Biases.
 26. TANIGUCHI, Hidetaka; SATO, Hiroshi; SHIRAKAWA, Tomohiro. A machine learning model with human cognitive biases capable of learning from small and biased datasets. In: 2018, vol. 8. Available from DOI: [doi:10.1038/s41598-018-25679-z](https://doi.org/10.1038/s41598-018-25679-z).
 27. MAREŠ, J. *Klasifikace spamu pomocí metody LSNB*. Bakalářská práce. VŠE Praha, 2020.
 28. *Stemming? Lemmatization? What?* [Online]. 2018 [visited on 2021-03-28]. Available from: <https://towardsdatascience.com/stemming-lemmatization-what-ba782b7c0bd8>.
 29. *Breast Cancer Wisconsin (Original) Data Set* [online] [visited on 2021-04-05]. Available from: [https://archive.ics.uci.edu/ml/datasets/breast+cancer+wisconsin+\(original\)](https://archive.ics.uci.edu/ml/datasets/breast+cancer+wisconsin+(original)).
-

30. CHATTERJEE, Shubrashankh. *Deep learning unbalanced training data? Solve it like this*. [Online]. 2018 [visited on 2021-05-25]. Available from: <https://towardsdatascience.com/deep-learning-unbalanced-training-data-solve-it-like-this-6c528e9efea6>.
31. *SpamAssassin Public Corpus* [online]. 2003 [visited on 2021-04-17]. Available from: <https://spamassassin.apache.org/old/publiccorpus/>.
32. *lsnb* [<https://github.com/jimrs/lsnb>]. GitHub, 2020.

Appendices

A. Full results of experiment 1 - Balanced training set

Accuracy	Alpha	Enhancement	Model	50	100	200	300	400	500	600	Avg
LSNN	0.5	0.1	save_node_unified	81.00%	91.83%	88.50%	96.00%	95.83%	94.67%	97.50%	92.19%
	0.5	0.05	value_node_unified	90.83%	90.00%	92.33%	96.00%	95.17%	96.00%	96.17%	93.79%
	0.6	0.05	save_node_unified	87.17%	90.83%	91.33%	96.00%	93.67%	97.33%	96.83%	93.31%
	0.6	0.05	save_node_unified_flattened	88.50%	93.83%	90.33%	92.67%	97.00%	95.50%	97.17%	93.57%
LSNB				94.50%	92.33%	94.00%	92.33%	95.00%	92.50%	93.50%	93.45%
eLSNB				80.67%	83.33%	87.67%	89.50%	88.33%	87.33%	88.67%	86.50%
NN				93.83%	94.50%	94.83%	97.50%	97.17%	97.67%	97.00%	96.07%

Figure A.1: Experiment 1 - Accuracy per training set size, source: author

F-Measure	Alpha	Enhancement	Model	50	100	200	300	400	500	600	Avg
LSNN	0.5	0.1	save_node_unified	83.24%	91.57%	88.36%	95.97%	95.74%	94.41%	97.46%	92.39%
	0.5	0.05	value_node_unified	90.37%	89.58%	91.99%	95.93%	94.99%	95.92%	96.10%	93.55%
	0.6	0.05	save_node_unified	87.91%	90.85%	91.42%	95.92%	93.33%	97.35%	96.77%	93.37%
	0.6	0.05	save_node_unified_flattened	88.96%	93.76%	90.27%	92.09%	96.97%	95.58%	97.12%	93.54%
LSNB				94.18%	91.73%	93.88%	91.81%	94.79%	91.95%	93.19%	93.08%
eLSNB				76.03%	80.00%	85.98%	88.27%	86.84%	85.55%	87.22%	84.27%
NN				93.54%	94.49%	94.72%	97.45%	97.12%	97.63%	96.93%	95.98%

Figure A.2: Experiment 1 - F-Measure per training set size, source: author

Precision	Alpha	Enhancement	Model	50	100	200	300	400	500	600	Avg
LSNN	0.5	0.1	save_node_unified	74.47%	94.66%	89.42%	96.62%	97.91%	99.26%	98.97%	93.05%
	0.5	0.05	value_node_unified	95.20%	93.48%	96.35%	97.59%	98.57%	97.92%	97.92%	96.72%
	0.6	0.05	save_node_unified	83.09%	90.70%	90.52%	97.92%	98.52%	96.71%	98.62%	93.72%
	0.6	0.05	save_node_unified_flattened	85.54%	94.88%	90.88%	100.00%	97.96%	93.89%	98.63%	94.54%
LSNB				100.00%	99.61%	95.83%	98.47%	98.91%	99.23%	97.80%	98.55%
eLSNB				100.00%	100.00%	99.56%	100.00%	99.57%	99.56%	100.00%	99.81%
NN				98.17%	94.65%	96.86%	99.31%	98.63%	99.31%	99.30%	98.03%

Figure A.3: Experiment 1 - Precision per training set size, source: author

Recall	Alpha	Enhancement	Model	50	100	200	300	400	500	600	Avg
LSNN	0.5	0.1	save_node_unified	94.33%	88.67%	87.33%	95.33%	93.67%	90.00%	96.00%	92.19%
	0.5	0.05	value_node_unified	86.00%	86.00%	88.00%	94.33%	91.67%	94.00%	94.33%	90.62%
	0.6	0.05	save_node_unified	93.33%	91.00%	92.33%	94.00%	88.67%	98.00%	95.00%	93.19%
	0.6	0.05	save_node_unified_flattened	92.67%	92.67%	89.67%	85.33%	96.00%	97.33%	95.67%	92.76%
LSNB				89.00%	85.00%	92.00%	86.00%	91.00%	85.67%	89.00%	88.24%
eLSNB				61.33%	66.67%	75.67%	79.00%	77.00%	75.00%	77.33%	73.14%
NN				89.33%	94.33%	92.67%	95.67%	95.67%	96.00%	94.67%	94.05%

Figure A.4: Experiment 1 - Recall per training set size, source: author

B. Full results of experiment 2 - Imbalanced training set (real ratio)

Accuracy	Alpha	Enhancement	Model	50	100	200	300	400	500	600	Avg
LSNN	0.5	0.1	save_node_unified	85.67%	91.67%	95.67%	92.83%	93.67%	95.17%	94.17%	92.69%
	0.5	0.05	value_node_unified	85.33%	92.83%	89.17%	91.50%	95.17%	94.17%	95.83%	92.00%
	0.6	0.05	save_node_unified	93.67%	91.17%	92.67%	91.00%	92.50%	94.17%	95.50%	92.95%
	0.6	0.05	save_node_unified_flattened	86.17%	90.50%	94.00%	93.83%	94.00%	94.50%	95.67%	92.67%
LSNB				90.50%	93.17%	94.33%	94.83%	94.50%	92.50%	93.83%	93.38%
eLSNB				85.00%	80.33%	90.50%	84.33%	88.50%	88.67%	89.50%	86.69%
NN				94.00%	95.50%	96.50%	96.67%	96.00%	95.33%	95.83%	95.69%

Figure B.1: Experiment 2 - Accuracy per training set size, source: author

F-Measure	Alpha	Enhancement	Model	50	100	200	300	400	500	600	Avg
LSNN	0.5	0.1	save_node_unified	86.73%	91.55%	95.62%	92.67%	93.26%	94.97%	93.91%	92.68%
	0.5	0.05	value_node_unified	85.19%	92.36%	88.12%	90.71%	94.92%	93.91%	95.68%	91.56%
	0.6	0.05	save_node_unified	93.43%	91.03%	92.11%	90.11%	92.17%	93.89%	95.37%	92.59%
	0.6	0.05	save_node_unified_flattened	87.59%	89.62%	93.73%	93.59%	93.64%	94.22%	95.49%	92.55%
LSNB				89.50%	92.69%	94.04%	94.59%	94.22%	92.12%	93.47%	92.95%
eLSNB				82.35%	75.52%	89.54%	81.42%	87.01%	87.22%	88.31%	84.48%
NN				93.86%	95.32%	96.39%	96.55%	95.83%	95.10%	95.68%	95.53%

Figure B.2: Experiment 2 - F-Measure per training set size, source: author

Precision	Alpha	Enhancement	Model	50	100	200	300	400	500	600	Avg
LSNN	0.5	0.1	save_node_unified	80.75%	92.81%	96.60%	94.77%	99.62%	98.92%	98.18%	94.52%
	0.5	0.05	value_node_unified	86.05%	98.86%	97.57%	100.00%	100.00%	98.18%	99.28%	97.14%
	0.6	0.05	save_node_unified	97.12%	92.44%	99.61%	100.00%	96.36%	98.53%	98.23%	97.47%
	0.6	0.05	save_node_unified_flattened	79.40%	98.80%	98.18%	97.47%	99.62%	99.26%	99.64%	96.05%
LSNB				100.00%	99.62%	99.26%	99.27%	99.26%	97.05%	99.25%	99.10%
eLSNB				100.00%	100.00%	99.59%	100.00%	100.00%	100.00%	99.58%	99.88%
NN				96.15%	99.28%	99.64%	100.00%	100.00%	100.00%	99.28%	99.19%

Figure B.3: Experiment 2 - Precision per training set size, source: author

Recall	Alpha	Enhancement	Model	50	100	200	300	400	500	600	Avg
LSNN	0.5	0.1	save_node_unified	93.67%	90.33%	94.67%	90.67%	87.67%	91.33%	90.00%	91.19%
	0.5	0.05	value_node_unified	84.33%	86.67%	80.33%	83.00%	90.33%	90.00%	92.33%	86.71%
	0.6	0.05	save_node_unified	90.00%	89.67%	85.67%	82.00%	88.33%	89.67%	92.67%	88.29%
	0.6	0.05	save_node_unified_flattened	97.67%	82.00%	89.67%	90.00%	88.33%	89.67%	91.67%	89.86%
LSNB				81.00%	86.67%	89.33%	90.33%	89.67%	87.67%	88.33%	87.57%
eLSNB				70.00%	60.67%	81.33%	68.67%	77.00%	77.33%	79.33%	73.48%
NN				91.67%	91.67%	93.33%	93.33%	92.00%	90.67%	92.33%	92.14%

Figure B.4: Experiment 2 - Recall per training set size, source: author

C. Full results of experiment 3 - Imbalanced training set (extreme ratio)

Accuracy	Alpha	Enhancement	Model	50	100	200	300	400	500	600	Avg
LSNN	0.5	0.1	save_node_unified	68.00%	75.50%	93.17%	83.00%	95.83%	90.33%	95.50%	85.90%
	0.5	0.05	value_node_unified	68.50%	78.50%	81.00%	91.83%	85.67%	91.33%	93.83%	84.38%
	0.6	0.05	save_node_unified	64.83%	81.67%	92.67%	83.00%	95.33%	93.67%	95.33%	86.64%
	0.6	0.05	save_node_unified_flattened	66.50%	77.00%	93.33%	84.67%	95.00%	90.50%	94.17%	85.88%
	0.6	0.2	save_node_unified_flattened	67.00%	76.83%	95.00%	85.17%	95.67%	95.67%	94.00%	87.05%
LSNB				68.00%	92.67%	90.00%	91.17%	90.33%	91.83%	88.50%	87.50%
eLSNB				82.33%	78.50%	84.00%	80.33%	82.00%	85.33%	88.00%	82.93%
NN				80.17%	74.00%	84.67%	86.50%	93.67%	92.50%	94.17%	86.52%

Figure C.1: Experiment 3 - Accuracy per training set size, source: author

F-Measure	Alpha	Enhancement	Model	50	100	200	300	400	500	600	Avg
LSNN	0.5	0.1	save_node_unified	54.07%	67.69%	92.87%	79.60%	95.78%	89.34%	95.38%	82.11%
	0.5	0.05	value_node_unified	54.24%	72.84%	76.54%	91.23%	83.27%	90.51%	93.47%	80.30%
	0.6	0.05	save_node_unified	45.76%	77.73%	92.20%	79.76%	95.21%	93.24%	95.17%	82.72%
	0.6	0.05	save_node_unified_flattened	49.62%	70.13%	93.10%	81.96%	94.90%	89.50%	93.89%	81.87%
	0.6	0.2	save_node_unified_flattened	51.47%	69.98%	94.83%	82.65%	95.58%	95.55%	93.77%	83.40%
LSNB				52.94%	92.57%	89.09%	90.42%	89.38%	91.17%	87.05%	84.66%
eLSNB				78.54%	72.61%	81.03%	75.62%	78.14%	82.88%	86.42%	79.32%
NN				75.46%	64.86%	81.89%	84.45%	93.26%	91.92%	93.83%	83.67%

Figure C.2: Experiment 3 - F-Measure per training set size, source: author

Precision	Alpha	Enhancement	Model	50	100	200	300	400	500	600	Avg
LSNN	0.5	0.1	save_node_unified	95.76%	99.35%	97.09%	99.50%	96.93%	99.59%	97.89%	98.02%
	0.5	0.05	value_node_unified	99.12%	98.86%	100.00%	98.46%	100.00%	100.00%	99.25%	99.38%
	0.6	0.05	save_node_unified	100.00%	98.97%	98.48%	98.53%	97.89%	100.00%	98.57%	98.92%
	0.6	0.05	save_node_unified_flattened	100.00%	100.00%	96.43%	99.52%	96.88%	100.00%	98.53%	98.77%
	0.6	0.2	save_node_unified_flattened	97.22%	99.39%	98.21%	99.53%	97.57%	98.24%	97.48%	98.23%
LSNB				100.00%	93.84%	98.00%	98.81%	99.19%	99.22%	99.57%	98.37%
eLSNB				100.00%	100.00%	99.51%	99.46%	99.48%	99.53%	99.57%	99.65%
NN				98.92%	100.00%	100.00%	99.55%	99.62%	99.61%	99.63%	99.62%

Figure C.3: Experiment 3 - Precision per training set size, source: author

Recall	Alpha	Enhancement	Model	50	100	200	300	400	500	600	Avg
LSNN	0.5	0.1	save_node_unified	37.67%	51.33%	89.00%	66.33%	94.67%	81.00%	93.00%	73.29%
	0.5	0.05	value_node_unified	37.33%	57.67%	62.00%	85.00%	71.33%	82.67%	88.33%	69.19%
	0.6	0.05	save_node_unified	29.67%	64.00%	86.67%	67.00%	92.67%	87.33%	92.00%	74.19%
	0.6	0.05	save_node_unified_flattened	33.00%	54.00%	90.00%	69.67%	93.00%	81.00%	89.67%	72.90%
	0.6	0.2	save_node_unified_flattened	35.00%	54.00%	91.67%	70.67%	93.67%	93.00%	90.33%	75.48%
LSNB				36.00%	91.33%	81.67%	83.33%	81.33%	84.33%	77.33%	76.48%
eLSNB				64.67%	57.00%	68.33%	61.00%	64.33%	71.00%	76.33%	66.10%
NN				61.00%	48.00%	69.33%	73.33%	87.67%	85.33%	88.67%	73.33%

Figure C.4: Experiment 3 - Recall per training set size, source: author